

AI agent adoption

This guidance provides a structured framework to help organizations successfully adopt AI agents as part of their broader [AI adoption strategy](#). It addresses the unique considerations that AI agents introduce. The series highlights Microsoft 365 agents and guidance for building custom agents using Microsoft Foundry and Microsoft Copilot Studio. It also includes strategies for designing an organization-wide data architecture to support AI agents at scale.

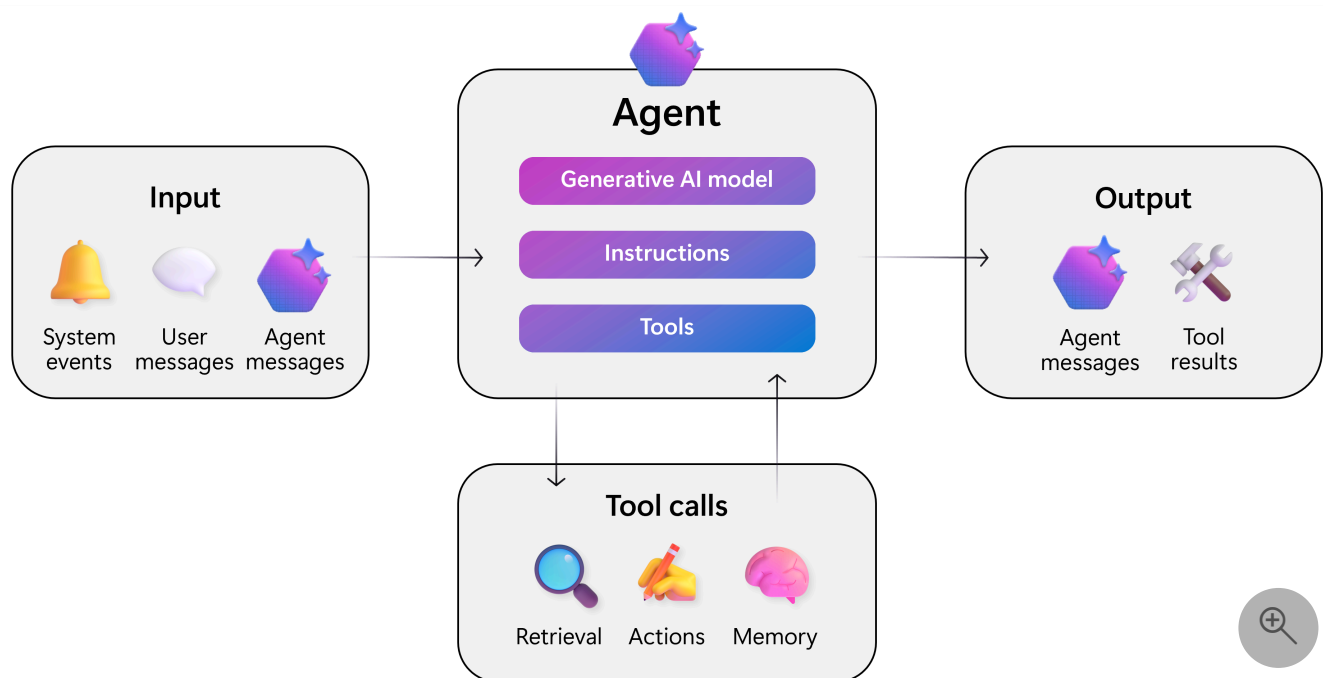
Through this guidance, leaders will gain actionable insights across four key areas: **(1) plan for agents**, **(2) govern and secure agents**, **(3) build agents**, and **(4) operate agents** (see *figure 1*).



Figure 1. Microsoft's AI agent adoption process.

What is an AI agent?

An AI agent is a flexible software program that uses generative AI models to interpret inputs, such as system events, user messages, or other agent messages, reason through problems, and decide on the most appropriate actions. Unlike traditional applications that rely on fixed rules, agents dynamically orchestrate workflows based on real-time context. This adaptability enables them to manage ambiguity and complexity that deterministic software cannot. Agents are built on five core components:



- **Generative AI model** serves as the agent's reasoning engine. It processes instructions, integrates tool calls, and generates outputs, either as messages to other agents or as

actionable results.

- **Instructions** define the scope, boundaries, and behavioral guidelines for the agent. Clear instructions prevent scope creep and ensure the agent adheres to business rules.
- **Retrieval** provides the grounding data and context required for accurate responses. Access to relevant, high-quality data is critical for reducing hallucinations and ensuring relevance.
- **Actions** are the functions, APIs, or systems the agent uses to perform tasks. Tools transform the agent from a passive information retriever into an active participant in business processes.
- **Memory** stores conversation history and state. Memory ensures continuity across interactions, allowing the agent to handle multi-turn conversations and long-running tasks effectively.

Difference from retrieval-augmented generation (RAG)

Standard RAG applications follow a deterministic retrieval process to answer queries. AI agents use a generative model to decide which knowledge and tools to use at each step. This adaptive approach enables multi-step reasoning and complex problem-solving, but it also introduces nondeterministic behavior that requires robust testing and governance.

For technical definitions, see [What is an agent?](#) and [what is a workflow?](#).

Why AI agents?

Adopting AI agents drives specific organizational outcomes. Understanding these benefits helps justify investment and prioritize use cases.

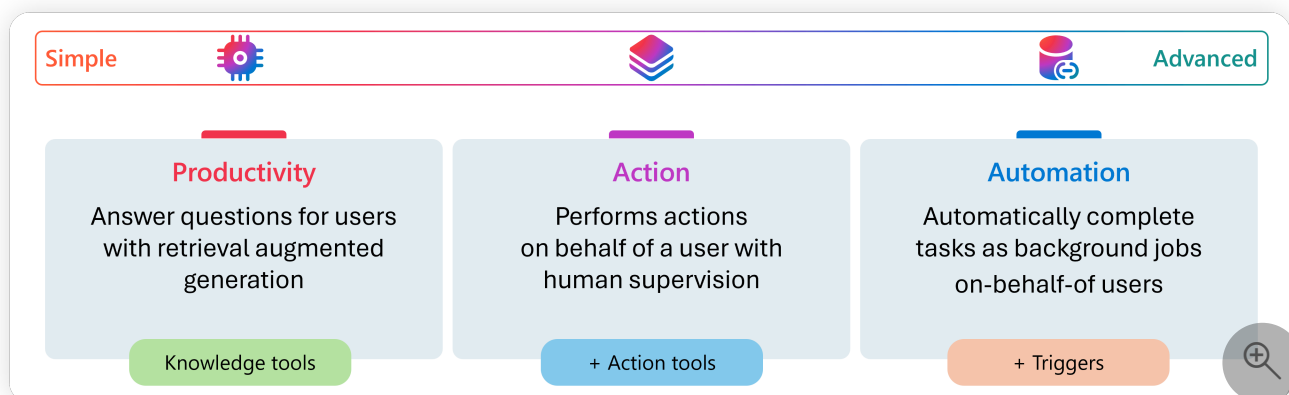
- **Efficiency:** Agents automate repetitive, low-value tasks. It reduces manual effort and operational costs, allowing resources to focus on strategic initiatives.
- **Speed:** Agents can process information and execute decisions fast, which can improve service delivery times and responsiveness to market changes.
- **Scalability:** Agents handle fluctuating workloads, and this elasticity supports growth and seasonal demand spikes.

These benefits lead to measurable results such as lower operating costs, improved customer satisfaction, and faster innovation. For leaders, this means AI agents aren't just a technology

investment. They're a strategic lever for growth and competitiveness. See [Business plan for AI agents](#) for more business rationale and use cases.

Agent types

Organizations typically deploy three categories of agents. Each category offers a different level of autonomy and business impact.



1. **Productivity agents.** These agents focus on information retrieval and synthesis to accelerate decision-making. They use knowledge tools to draw data from various sources and retrieve it for the user. This capability increases employee accuracy and reduces time spent searching for information in scenarios like customer service support and internal knowledge management.
2. **Action agents.** These agents perform specific tasks within defined workflows, such as updating records or triggering processes. They use knowledge tools combined with action tools to accomplish tasks. This approach streamlines operations and reduces manual data entry errors in use cases like service ticket creation and system monitoring.
3. **Automation agents.** These agents manage complex, multi-step processes with minimal oversight. They use knowledge tools and action tools, plus triggers that determine when to run, stop, or escalate an issue. This autonomy enables scalable automation for scenarios like supply chain optimization, though it requires rigorous governance to manage the increased complexity.

Next steps

To realize the potential of AI agents, align the adoption strategy with specific business outcomes. The following sections explore how to create internal and customer-facing impact and guide teams to use agents effectively.

Last updated on 12/04/2025

Business plan for AI agents

This article explains how to develop a business plan for AI agents that aligns your investment with organizational priorities. It guides you through identifying high-value use cases, ranking them by impact and feasibility, and defining success metrics to measure return on investment. Building this plan is the first step in the **Plan for agents** phase of AI agent adoption (*see figure 1*). Without a well-defined plan, organizations risk wasting time and resources on use cases that are poorly suited for AI agents.

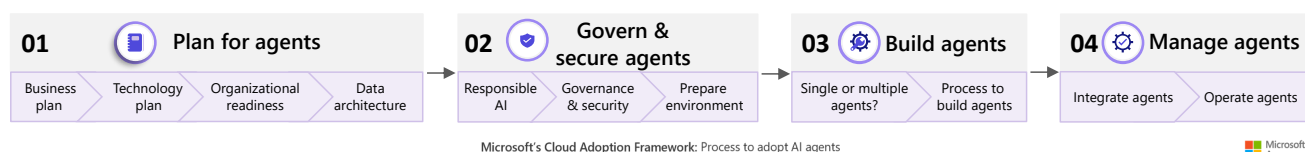


Figure 1. Microsoft's AI agent adoption process.

When not to use AI agents

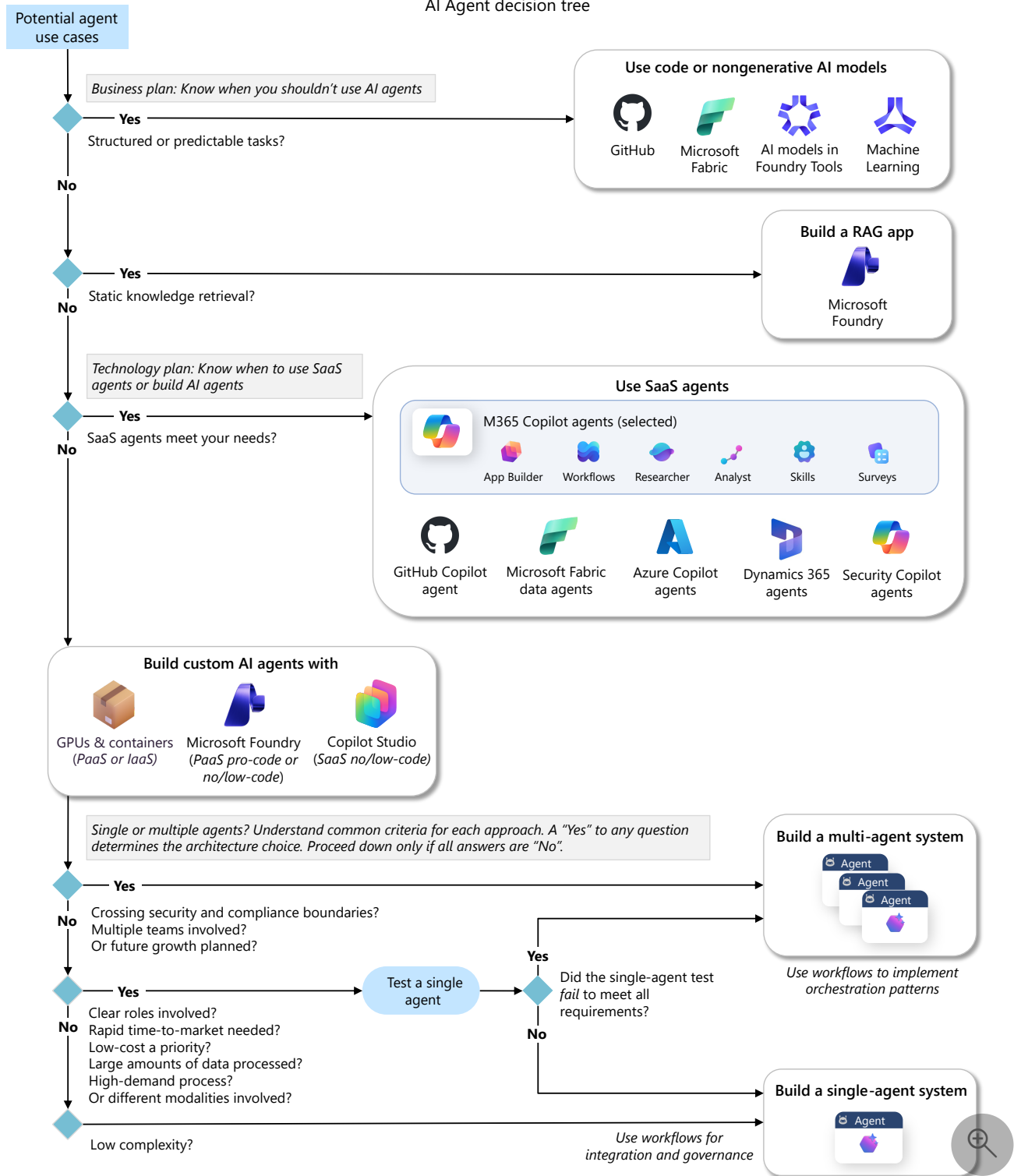
The first step in identifying high-value agent use cases is understanding when *not* to use an agent. Agents introduce nondeterministic behavior, latency, and cost that are unnecessary for many scenarios. By eliminating unsuitable use cases early, you narrow the focus to opportunities where agents deliver measurable business value. This approach prevents wasted effort on projects better served by deterministic code or simpler AI solutions.

- **Structured and predictable tasks.** Use deterministic code or nongenerative AI models when the workflow is predictable, rule-based, and doesn't require reasoning. If a process follows a fixed path with well-defined inputs and outputs, deterministic code or nongenerative AI models is faster, cheaper, and more reliable.
- **Static knowledge retrieval.** Use standard Retrieval-Augmented Generation (RAG) when the goal is answering questions or generating content from a fixed index. If the workflow doesn't require tool execution or multi-step reasoning, an agent adds unnecessary complexity. Standard generative AI applications are sufficient for single-turn interactions where the system summarizes data or answers questions without orchestration. Examples include FAQ bots, document search with generative summaries, and knowledge base assistants. See [RAG](#).

Refer to the decision tree below to assess whether your use case is suitable for an agent. If you answer "No" to the first two questions, the process likely requires the reasoning and tool orchestration capabilities that agents provide.

Microsoft's Cloud Adoption Framework

AI Agent decision tree



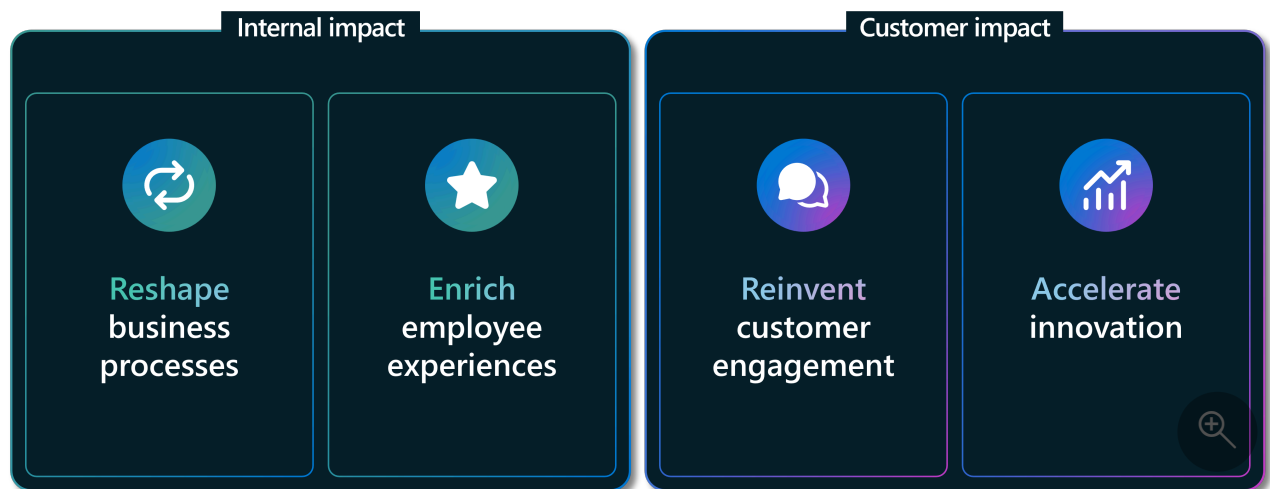
Microsoft facilitation:

For nongenerative AI solutions, see [Microsoft Fabric data science](#). See also the prebuilt speech, language, and translator models in [Foundry Tools](#). Build your own predictive models in [Azure Machine Learning](#).

When to use AI agents

After eliminating unsuitable scenarios, focus on opportunities where agents drive strategic value. Unlike deterministic software that follows a fixed path, agents reason over data and tools to formulate plans. See also [What is an AI agent?](#). To maximize return on investment, start by identifying high-impact business areas, then validate that the specific processes within those areas require the unique reasoning capabilities of an agent.

1. **Target strategic areas.** Direct agent development toward pillars that scale operations and drive competitive advantage. Aligning agent capabilities with these strategic goals ensures measurable business value.



- **Reshape business processes.** Automate complex, multi-step workflows such as supply chain adjustments or incident triage. This scales operations without linear increases in headcount.
 - **Enrich employee experiences.** Augment staff by handling cognitive load, such as synthesizing research or drafting technical content. This reduces cycle times and allows employees to focus on strategic decision-making.
 - **Reinvent customer engagement.** Resolve dynamic customer queries autonomously with context-aware responses. This improves resolution speed and customer satisfaction compared to rigid chatbots.
 - **Accelerate innovation.** Use agents to analyze market trends or simulate scenarios. This shortens product development cycles and enables faster experimentation.
2. **Select processes that require reasoning.** Within strategic areas, focus on processes where inputs vary significantly and outcomes depend on context rather than fixed rules. Agents add value when a task requires interpreting intent, reasoning through multiple steps, or selecting tools dynamically. These scenarios involve ambiguity that deterministic automation cannot handle effectively. Agents excel in the following types of tasks:
 - **Dynamic decision-making.** The process requires reasoning across multiple steps with conditional logic. The system must evaluate intermediate results and adjust its approach based on context rather than following a predetermined sequence. For

example, an agent triages support tickets by analyzing ticket content, checking system logs, and escalating to specialists only when automated resolution fails.

- **Complex orchestration.** The process chains multiple tools, APIs, or services together. The agent selects and sequences these tools based on the specific request and the results of prior actions rather than executing a static integration pattern. For example, an agent processes expense reports by extracting data from receipts, validating amounts against policy rules, querying approval workflows, and updating financial systems based on approval outcomes.
- **Adaptive behavior.** Inputs are ambiguous or variable, and the system must interpret intent and adjust its strategy accordingly. The agent formulates a plan that responds to the nuances of each request rather than applying the same process to every input. For example, an agent handles customer inquiries by interpreting vague requests, searching knowledge bases, checking order status, and generating context-specific responses that address the underlying need rather than the literal question.

If a task fits these general criteria, an agent provides measurable value over standard automation or generative AI alone. If the process follows predictable logic with consistent inputs and outputs, use deterministic code or nongenerative AI models instead.

3. **Validate value through rapid piloting.** Test the reasoning capabilities of an agent in a low-code environment before investing in custom code. Platforms like Microsoft Copilot Studio or Microsoft Foundry allow for rapid prototyping to verify that an agent can handle the required ambiguity. This step prevents over-engineering solutions that might use simpler automation.

Microsoft facilitation:

See the [Microsoft Scenario Library](#), [AI Use Cases catalog](#), and [Sample Solution Gallery](#) to benchmark internal ideas against proven patterns.

How to prioritize AI agent use cases

Not all agent initiatives deliver equal value. Prioritize use cases that align with strategic goals and demonstrate impact quickly. Use three criteria to evaluate and rank candidate use cases: business impact (value), technical feasibility (complexity), and user desirability (value). See the following image for a framework on prioritizing AI agent use cases. The guidance that follows walks you through each of the criteria in more detail.

Business impact		
No alignment 1	Executive strategy alignment Does the use case align with the overall business strategy and goals?	Very aligned with board-level support 5
Value not understood yet 1	Business value How does the use case generate business value?	Value very clear and significant 5
Long roll-out and high user impact 1	Change management timeframe What is the expected time required to implement the use case and manage any changes?	Short roll-out and low user impact 5
User desirability		
Not fully known or defined yet 1	Key personas Who are the key stakeholders and users who will be impacted by the use case?	Clearly defined and understood 5
Low appeal or desire for a user 1	Value proposition What is the value proposition for the user? What is the potential appeal and adoption of the use case?	Highly desired by users 5
Expected high resistance by users 1	Change resistance What is the level of change resistance for the use case? How can change be managed?	Expected very low resistance to change 5
Technical feasibility		
Unknown currently 1	Implementation and operation risks What are the potential risks associated with the use case and plans for mitigation?	Well understood and addressable 5
Unknown currently 1	Sufficient safeguards Are the appropriate safeguards in place to manage risk and ensure compliance?	Mature security, RAI and compliance practices in place 5
Type of tech and why is not clear 1	Technology fit What is the fit of the use case with identified technologies?	Need and benefit of tech is clear and compelling 5

Evaluate business impact

Evaluate each use case across three dimensions using a 1–5 scale, where lower scores indicate unclear or weak business impact and higher scores indicate strong business impact:

- **Executive strategy alignment:** Confirm whether the use case directly supports organizational priorities. If it doesn't align with business strategy, it shouldn't proceed. The strongest candidates advance strategic objectives and have visible board-level sponsorship.
- **Business value:** Quantify the impact. Use cases with vague or unproven benefits should be deprioritized. Select initiatives that deliver measurable outcomes with clear evidence of significant value. Examples include reducing operational costs, increasing revenue, or improving customer satisfaction.
- **Change management timeframe:** Consider the expected time and effort required to implement the use case and manage associated changes. A lengthy rollout with significant user disruption signals a challenging implementation. A short deployment cycle with minimal impact on users indicates strong feasibility and readiness.

Measure technical feasibility

Require a technical feasibility summary for each candidate use case. Include considerations such as data quality, system dependencies, integration challenges, and implementation timelines. Favor projects with short deployment cycles, minimal disruption, and strong compatibility with documented APIs or connectors. Evaluate each use case across three dimensions using a 1–5 scale, where lower scores indicate unclear or weak technical feasibility and higher scores indicate strong technical feasibility:

- **Implementation and operation risks.** Identify and address risks upfront. If risks are unknown or mitigation plans are absent, the use case shouldn't advance. Prioritize scenarios where risks are well understood and mitigation strategies are documented and actionable.
- **Sufficient safeguards:** Validate compliance and security measures. Lack of safeguards or unclear governance creates unacceptable exposure. Select use cases backed by mature security controls, responsible AI practices, and regulatory compliance frameworks.
- **Technology fit:** Confirm compatibility with existing systems. If the technology requirements are unclear or poorly aligned, integration is more likely to fail. Favor solutions where the technology choice is justified, the benefits are compelling, and integration with current infrastructure is straightforward.

Measure user desirability

Gather evidence through interviews or surveys to validate pain points and openness to change. Prioritize projects with strong user advocates and minimal resistance, as AI agents succeed only when people use them consistently and trust their output. Evaluate each use case across three dimensions using a 1–5 scale, where lower scores indicate unclear or weak desirability and higher scores indicate strong desirability:

- **Key personas:** Assess whether the key stakeholders and users affected by the use case are clearly identified. A low score means these personas aren't well understood or defined. A high score means they're clearly defined and their roles are well understood.
- **Value proposition:** Consider the appeal and adoption potential of the use case for users. A low score reflects minimal perceived value or low interest. A high score indicates the solution is highly desired and offers clear benefits to users.
- **Change resistance:** Evaluate the expected level of resistance to adopting the solution. A low score suggests significant resistance and challenges in managing change. A high score indicates low resistance and strong readiness for adoption.

Define success metrics

Establish measurable success criteria before development begins to ensure that agent adoption aligns with strategic business goals. Without clear metrics, organizations cannot validate whether an agent delivers the intended value or justifies the investment. These criteria serve as the benchmark for future management and measurement phases, enabling teams to track performance against initial objectives and make data-driven decisions about scaling, refining, or retiring solutions.

1. **Set baseline business goals.** Identify the key performance indicators (KPIs) that the AI agent must improve. For existing processes, measure current performance to establish a baseline. This baseline comparison enables accurate tracking of post-deployment impact. For new processes or early-stage businesses, estimate initial performance targets and refine these targets as operations mature.
2. **Use business metrics as decision gates.** Apply success criteria throughout the development lifecycle to guide investment decisions. Use these success metrics as checkpoints to determine whether the project continues, pivots, or stops. If a pilot fails to meet predefined benchmarks, reassess the use case or terminate the pilot initiative to avoid unnecessary cost and effort.
3. **Evaluate post-deployment performance.** Continue to measure success after integration. Compare actual results against target KPIs to determine whether the AI agent delivers expected value. If the AI agent underperforms, use the performance data to decide whether to refine the solution, retire the agent, or redirect resources to more promising opportunities.

This structured evaluation approach ensures that every AI agent initiative remains accountable to business value and supports continuous improvement across the portfolio.

Next step

Technology plan

Last updated on 12/05/2025

Technology plan for AI agents

This article provides guidance on how to select the right technology platform for each of your potential agent use cases. It covers adopting a ready-to-use SaaS agent or building a custom agent with Microsoft Foundry (PaaS) or Microsoft Copilot Studio (SaaS). Developing a technology plan is the second step in the **Plan for agents** phase of AI agent adoption (see *figure 1*).

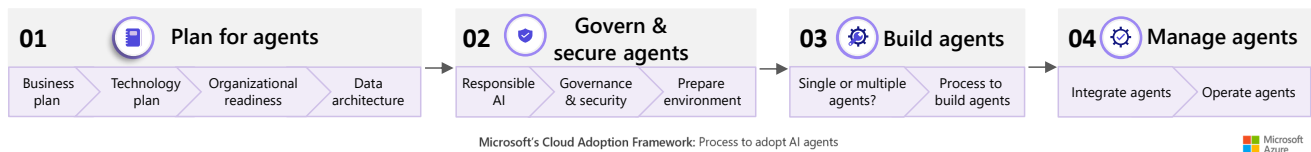


Figure 1. Microsoft's AI agent adoption process.

Effective technology adoption aligns goals with cost, level of effort, and customization needs. This alignment matches the technology to the use case and balances the effort required to achieve a return on investment. Understanding the available landscape ensures the right choice between adopting a ready-to-use SaaS agent or building a custom solution to provide business advantage.

AI agent decision tree

The AI agent decision tree guides the technology selection process by focusing on one primary question: **Does a SaaS agent meet your functional requirements?** If a SaaS agent satisfies your needs, adopt the prebuilt solution. If no SaaS agent fits the use case, you must build a custom agent. Determining which platform to use for a custom build, Microsoft Foundry, Microsoft Copilot Studio, or custom infrastructure, requires further investigation. The following sections provide guidance on selecting the right platform based on your specific requirements.

Microsoft's Cloud Adoption Framework

AI Agent decision tree

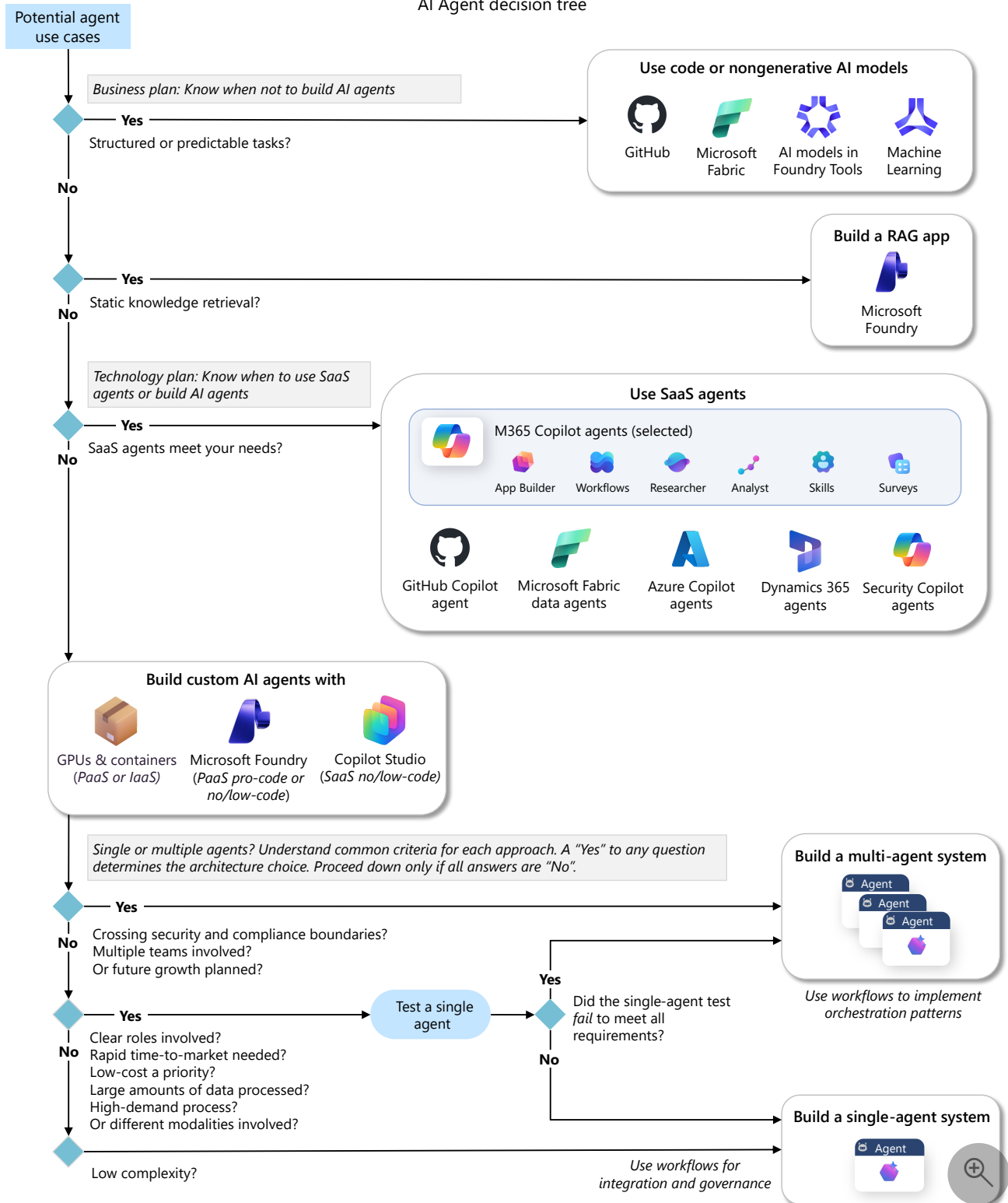


Figure 2. Microsoft's AI agent decision tree.

Use SaaS agents

SaaS agents are ready-to-use solutions built by Microsoft that enable immediate deployment. These agents provide rapid value for standard business functions but offer limited

customization compared to custom builds. Evaluate the agents available across the technology stack to determine if a prebuilt solution meets your requirements.

- **Agents in Microsoft 365 Copilot.** [App Builder agent](#), [Workflows agent](#), and [Researcher](#) enable task automation and information synthesis from Microsoft 365 data. Use the [Agent Success Kit](#) to structure your deployment approach and governance model.
- **In-product SaaS agents.** [GitHub Copilot agents](#) support coding tasks. [Microsoft Fabric data agents](#) enable data analysis from data in Fabric. [Azure Copilot agents](#) provide insights and recommendations for your Azure environment. [Dynamics 365 agents](#) help with customer service workflows. [Security Copilot agents](#) enhance threat detection and response. These agents deliver immediate capabilities for domain-specific productivity and process automation. For more context, see [Overview of available Copilots](#).

When no SaaS agents meet your functional requirements, build custom agents to achieve the necessary capabilities and integration depth.

Build AI agents

Choose a build path based on your organization's technical capabilities, timeline, and control requirements. Microsoft provides two primary platforms, Foundry (PaaS) and Copilot Studio (SaaS), with GPUs and containers (IaaS) as a third option.

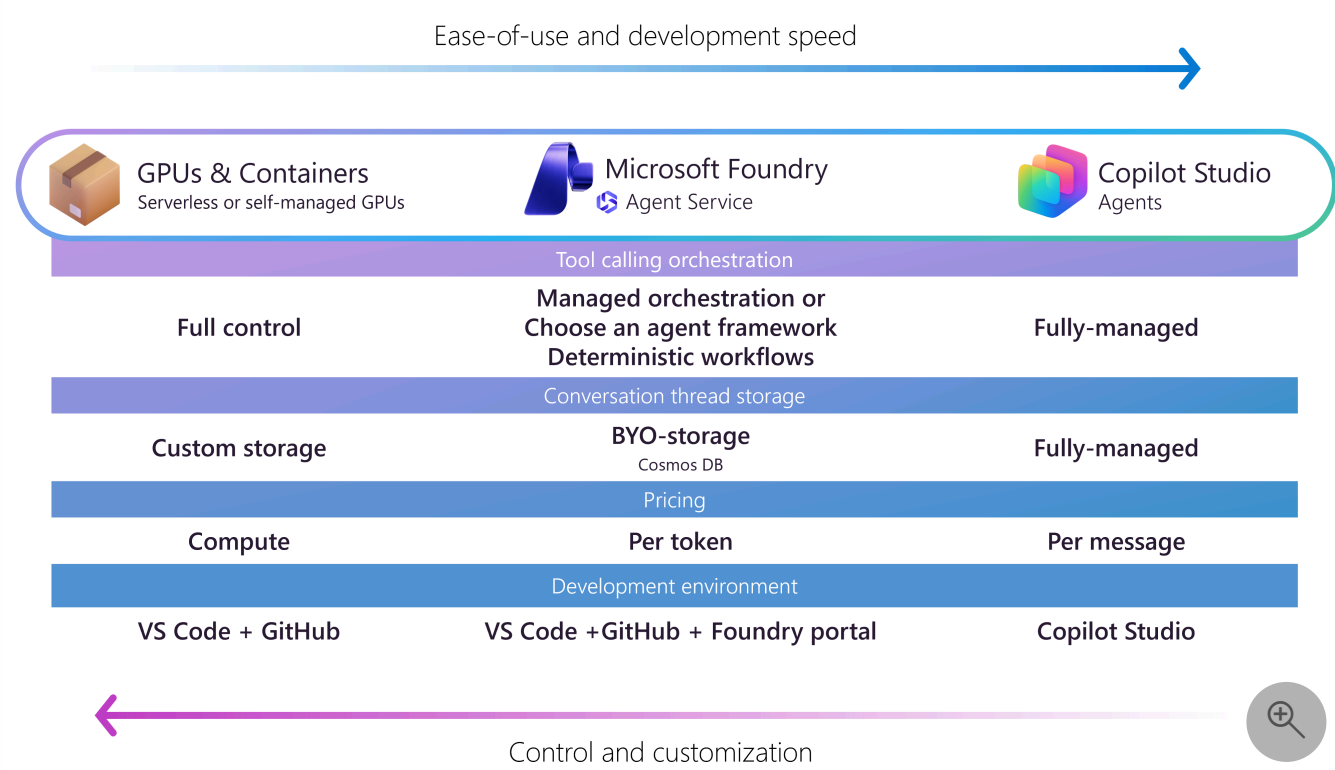


Figure 3. Microsoft's AI agent build options.

Microsoft Foundry

Microsoft Foundry provides a platform-as-a-service (PaaS) environment for no/low-code agent development (declarative agents) or pro-code agent development (hosted agents). Foundry lets you pick your own models, and it consolidates runtime, orchestration, and integration into a managed environment. You don't need to manage the underlying virtual machine or Kubernetes infrastructure. See the following image for more capabilities:

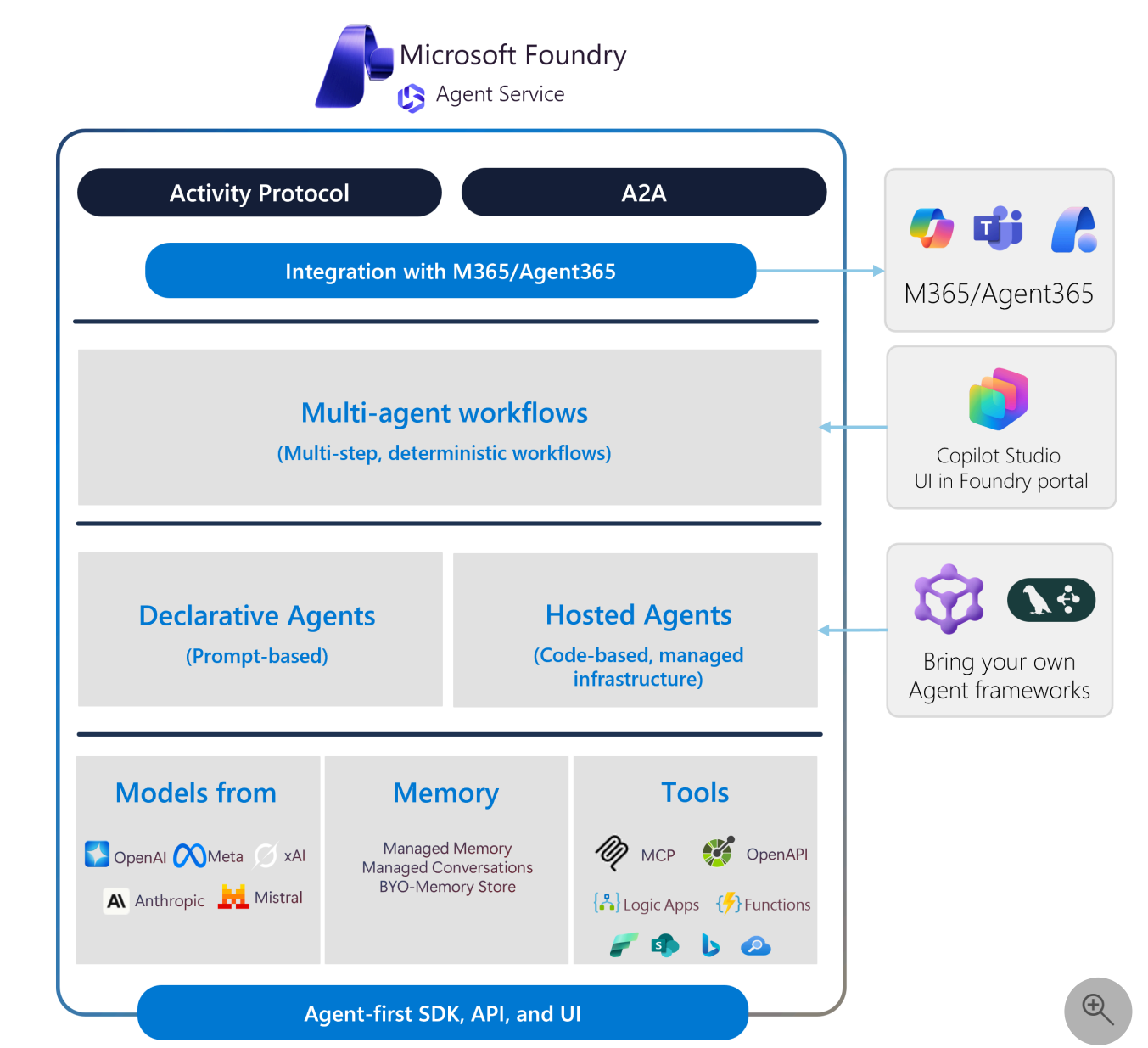


Figure 4. Overview of Microsoft Foundry's AI capabilities.

- **Activity protocol, A2A, & integration with M365/Agent 365.** Foundry supports the [Activity Protocol](#) and agent-to-agent (A2A) patterns for standardized messaging. Agents can be published to [Microsoft 365](#) and [Agent 365](#) to surface capabilities directly in user workflows.
- **Multi-agent workflows.** [Workflows](#) orchestrate complex business processes by handling sequential logic, conditional branching, and state management across multiple agents.

- **Declarative agents.** [Prompt-based agents](#) rely primarily on model reasoning and instructions, simplifying updates and versioning for behavior-driven agents.
- **Hosted agents.** Code-first, [hosted agents](#) support custom libraries or frameworks. This option provides a managed runtime that handles provisioning and scaling while allowing full code control.
- **Models from.** The [Model catalog](#) includes models from OpenAI, Anthropic, Meta, and Mistral, allowing selection based on specific performance, latency, and cost requirements.
- **Memory.** [Managed memory](#) maintains conversation context. For strict data sovereignty requirements, a bring-your-own (BYO) memory store option is available.
- **Tools.** The [Tool catalog](#), model context protocol (MCP), and OpenAPI specifications enable connections to external systems. Integration with Azure Logic Apps and Azure Functions supports serverless automation.

Setup options: Foundry has two setup options that align with your security and operational needs. The [basic setup](#) is for rapid prototyping and individual development when speed and ease of access are priorities. This option lacks network isolation. The [standard setup](#) provides gain fine-grained control over data, security, and networking. Within the standard setup, you can use public networking for nonconfidential workloads that require enterprise data controls. You can also use private networking for confidential workloads that must integrate with existing Azure resources to meet strict compliance standards. Review the [comparison and deployment guide](#) for details on both topologies.

Foundry playground: The [Foundry playground](#) enables you to build and test prototypes. Follow the quickstart guide to [create a new agent](#).

Microsoft Copilot Studio

[Microsoft Copilot Studio](#) offers a software-as-a-service (SaaS) platform for low-code development. It enables business teams to deploy agents quickly with moderate customization. The platform includes prebuilt connectors, supports retrieval and task agents, and integrates with Azure AI Search. Built-in responsible AI features reduce the need for custom safeguards.

Use Copilot Studio's low-code interface with Foundry's advanced models to handle sophisticated use cases while maintaining SaaS security and reliability. This hybrid approach allows business teams to build agents without extensive coding while accessing enterprise-grade AI capabilities. The combination reduces development time compared to full pro-code solutions while providing more customization than standard SaaS agents.

Test use cases with the [60-day free trial](#) before committing to production deployment. Review available [access options](#) to determine the best entry point for your organization.

GPUs and containers

You can host the AI model that an agent uses on GPU-backed containers. This approach is ideal when you want to run your own models rather than relying on Foundry endpoints. In this setup, the AI model runs on GPUs while the agent itself runs on CPUs. When configuring your agent and specifying the model it will use, you point it to the hosted model. The choice of compute platform depends on the size of the model and the level of control you need over the environment.

For hosting **AI models**, Azure Container Apps with serverless GPU support is a good option for small language models because it automatically handles scaling and minimizes operational overhead. For large language models, Azure Kubernetes Service is recommended since it supports large GPU clusters, offers higher throughput, and provides advanced control over scaling and networking.

For hosting **AI agents**, you can use either Azure Kubernetes Service or Azure Container Apps with CPU-based clusters to run agents with a custom orchestration framework.

Validate technology choices

Organizations often use multiple approaches to meet diverse requirements. Validate platform fit through structured experimentation before scaling.



1. **Run time-boxed experiments.** Build short prototypes for each candidate solution. Allocate one to two weeks per option. Compare low-code agents in Copilot Studio with pro-code solutions in Microsoft Foundry. Evaluate development speed, functional coverage, and integration complexity.
2. **Require documentation and stakeholder review.** Document findings and present clear recommendations. If a low-code solution meets functional and security requirements, proceed with that option. If not, shift to pro-code or adjust scope. Stakeholder review reduces rework and increases confidence.
3. **Assess single-agent versus multi-agent architecture.** Use prototypes to determine whether the task requires multiple specialized agents or a single agent. Avoid unnecessary complexity. If a single agent meets business needs efficiently, proceed with

that approach. If not, define a roadmap for multi-agent orchestration. Refer to [Single agent or multiple agents?](#).

 [Expand table](#)

Solution	Approach	Agent types	Best for
SaaS agents	Ready-to-use (SaaS)	Retrieval, Task	Personal productivity. Requires minimal customization to deliver immediate value.
Microsoft Foundry	Pro-code and low/no-code (PaaS)	Retrieval, Task, Autonomous	Strategic transformation. Supports deep integration and custom logic.
Microsoft Copilot Studio	Low/no-code (SaaS)	Retrieval, Task, Autonomous	Process transformation. Enables fast development with minimal coding and SaaS security.
GPUs and containers	Pro-code (PaaS or IaaS)	Retrieval, Task, Autonomous	Compliance-sensitive or highly customized (custom model configurations, private networking, strict isolation) workloads with full control of the entire technology stack.

See the general [AI decision tree](#) for more guidance.

Next step

After you define your business and technology strategies for AI agents, focus on organizational structure and talent. Your teams need the right skills and structure to deliver and sustain these solutions.

[Organizational readiness](#)

Last updated on 12/18/2025

Organizational readiness for AI agents

This article outlines strategies for organizational readiness, including platform governance, workload alignment, and AI Center of Excellence integration. Organizational readiness is the third step in the **Plan for agents** phase of AI agent adoption (see figure 1).



Figure 1. Microsoft's AI agent adoption process.

Organizational readiness aligns team structures and skills to support AI agent development. It establishes clear responsibilities and governance models that enable scalable innovation. Without this preparation, organizations risk isolated experiments, inconsistent security practices, and an inability to scale AI agents across the enterprise.

Define responsibilities for agent development

Successful AI agent adoption relies on integrating agent responsibilities into your existing operating model. This integration requires a clear distinction between the platform that provides the foundation and the workloads that deliver the application logic. The platform team focuses on governance and security at scale, while workload teams concentrate on business value and agility. An AI Center of Excellence (AI CoE) unifies these efforts through shared standards and expert guidance (see figure 2).



Figure 2. Typical AI agent responsibilities across an organization.

- **Platform responsibilities.** The platform team manages the technical foundation and governance guardrails. They must audit and enforce the [responsible AI policies](#) and [governance standards](#) the organization adopts. This centralization ensures observability,

compliance, and consistent risk management across the enterprise. For implementation guidance, see [prepare an agent environment](#).

- **Workload responsibilities.** Workload teams operate within business units and own the end-to-end lifecycle of specific agents. They define business requirements, curate domain-specific data, design conversation flows, and integrate agents into business processes. These teams inherit the security controls of the platform and must follow the approved [process to build agents](#).
- **AI Center of Excellence (AI CoE).** The AI CoE acts as a centralized advisory body that drives strategy and prevents fragmented adoption. It provides the technical expertise and consultation needed to scale AI initiatives successfully. The CoE embeds responsible AI principles into organizational policy, offers expert development guidance, and leads training efforts. For more information, see [Establish an AI CoE](#).

Gain agent skills and training

Organizations must identify the skills required to support AI agents and address gaps through training or hiring. Each project team needs access to the necessary expertise. Retrain existing staff where feasible, such as upskilling web developers to use Copilot Studio, and bring in specialists for advanced needs like ML engineers for complex Foundry projects. Common skill areas include:

 Expand table

Skill area	Description
Prompt engineering	Techniques for designing inputs, system instructions, and orchestration logic that guide model behavior effectively.
Agent optimization	Processes for fine-tuning models , evaluating response quality against ground truth, and monitoring performance metrics.
AI ethics and governance	Application of responsible AI principles to ensure agents adhere to safety, fairness, and compliance standards.
Data engineering for AI	Strategies for structuring unstructured data, managing vector indexes, and implementing Retrieval-Augmented Generation (RAG) patterns.
AI security	Methods for detecting and mitigating AI-specific threats, such as prompt injection and jailbreaks.

A structured training program builds AI competency across teams. Treat skills development as a core part of technology adoption rather than an afterthought.

1. **Use Microsoft's free training resources.** Use Microsoft Learn's free online modules and certifications, such as [Azure AI Engineer Associate](#), to give teams a grounding in AI services. See the [AI agents hub](#) for helpful resources. See also the [AI Skills Navigator](#).
2. **Run hands-on workshops.** Organize internal workshops or hackathons. For example, hold a prompt engineering lab where participants practice improving AI responses, or a hackathon where cross-functional teams prototype a simple agent. These activities build skills, enthusiasm, and idea-sharing.
3. **Consider partner-led training.** Bring in Microsoft or [certified training partners](#) for tailored sessions or bootcamps on specific tools like Foundry or Copilot Studio. Training on actual data accelerates learning through expert guidance.
4. **Encourage mentorship and peer learning.** Recognize employees who built AI agents as champions who can coach others. Create internal communities of practice, such as a Teams channel, for AI agent developers to ask questions and share tips.

Microsoft facilitation:



Foundry: [Training path](#). See the tutorial [Build and evaluate an enterprise agent](#).



Copilot Studio: [Training path](#).

Address AI change management

Preparing the organization for AI-driven change ensures long-term adoption. Manage expectations by communicating early about what AI agents can and can't do. Leadership must reinforce that adopting AI agents is a strategic priority. This encouragement helps managers and teams embrace new tools rather than resist them. Celebrate successes and recognize teams that deliver value with AI to foster a culture of innovation.

Next step

Data architecture

Last updated on 12/04/2025

Data architecture for AI agents

This article outlines how to prepare your data estate for AI agents, ensuring they operate on a unified, secure, and governed foundation. Because agents synthesize information rather than create it, their accuracy depends entirely on the quality and accessibility of underlying sources. Fragmented or ungoverned data leads to misleading results and security risks. This planning is the final step in the **Plan for agents** phase of AI agent adoption (*see figure 1*).

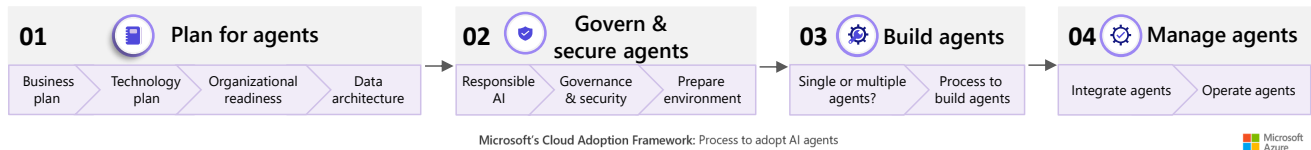


Figure 1. Microsoft's AI agent adoption process.

Use this guidance to prepare your data estate for Microsoft SaaS agents, including Microsoft 365 Copilot, and custom agents built on Microsoft Foundry and Microsoft Copilot Studio. These architectural decisions ensure agents operate on authoritative content, respect compliance boundaries, and support rapid deployment.

Conceptual data architecture

This architecture maps data domains to your organizational structure. It uses Microsoft Fabric OneLake as the central data lake. Each domain manages its own data, reports, and visualizations. Teams expose these datasets through Fabric IQ, Fabric Data Agents, or indexes in Microsoft Foundry and Microsoft Copilot Studio. This model ensures governance across the estate. Azure workloads and services also align to these data domains and ingest data into OneLake to enable analytics on those workloads.

Microsoft's Cloud Adoption Framework

Data architecture for AI agents

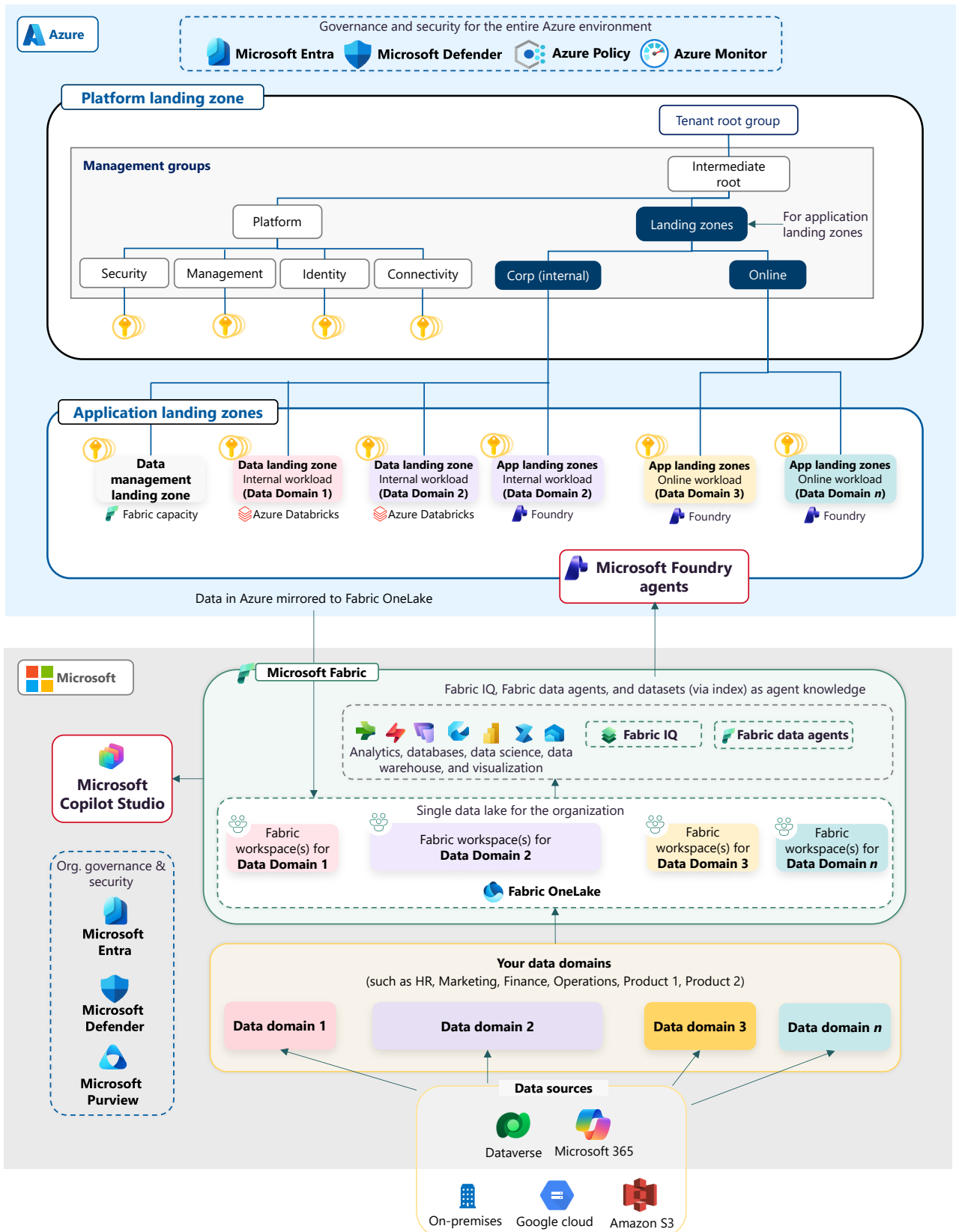


Figure 2. Conceptual data architecture for AI agents.

Prepare Microsoft 365 data

Microsoft 365 Copilot productivity agents, such as Researcher agent, App Builder agent, and Workflows agent, rely on organizational data stored in Microsoft Graph. Microsoft Graph includes data from SharePoint, OneDrive, Exchange, and other Microsoft 365 sources. These agents retrieve documents, emails, and collaboration artifacts while honoring existing permissions. See [Microsoft 365 Copilot architecture](#). Agents in Foundry ([SharePoint](#)) and [Copilot Studio](#) can also connect to data in Microsoft 365 apps. Use the [Microsoft 365 adoption guide](#) to get your data ready.

What data belongs in Microsoft 365? Keep content in Microsoft 365 when its primary purpose is collaboration, reference, or knowledge retrieval. Examples include policies, procedures, playbooks, templates, FAQs, emails, Teams chats, meeting transcripts, agendas, project documents, and departmental knowledge bases. These normally require document-level permissions in Word, Excel, PowerPoint, Outlook, and Teams.

Unify business data

Microsoft 365 provides collaboration and knowledge content for AI agents, but process automation and performance analytics require more than documents and emails. These agents need a unified, governed data foundation that spans operational systems and structured business data, and can include curated signals from Microsoft 365 content. Microsoft Fabric delivers this foundation through OneLake, a single, logical data lake automatically provisioned for every Fabric tenant. OneLake supports multiple analytical engines and uses Delta Parquet format for consistency and performance.

Unifying and cleansing data within each domain creates a reliable base for agent development. Clean, consistent data accelerates design and deployment and ensures analytics and automation produce accurate results. This approach reduces friction for teams and supports governance by default.

Why a single data platform matters

AI agents can technically operate without a unified platform by connecting directly to domain-specific systems or using federated queries. However, a unified approach delivers significant advantages:

 Expand table

Benefit	Impact
Lower integration effort	Agents connect to a single logical data lake rather than multiple systems, reducing complexity and accelerating development.

Benefit	Impact
Consistent governance	Policies, compliance controls, and security models are applied uniformly across all data domains.
Faster innovation	New agents and capabilities can be deployed without repeating integration work for every source.
Improved discoverability	Certified data products are easier to find and consume, enabling agents to reason over trusted datasets.
Better performance for AI	Retrieval strategies like RAG and MCP operate on optimized, governed data rather than fragmented sources.

Microsoft Fabric addresses these needs through OneLake, a single logical data lake where teams organize data by domain and publish certified data products. This approach preserves domain ownership while making data easier to consume and govern. It’s not just an analytics architecture. It’s a foundation for AI agents to operate effectively and securely.

What data should you unify?

Unify data that supports operational decisions, performance tracking, or automation. Start by asking a clear question: "Does this data help explain or measure a business process?" If the answer is yes, prepare that data for analytics and AI. If the answer is no, keep it in collaboration tools for reference and teamwork.

Examples of data you should unify include transactional records, customer interactions, inventory details, financial metrics, and curated signals extracted from collaboration content. These datasets provide measurable outcomes and enable agents to reason over structured information. Exclude content that serves only collaborative purposes, such as drafts, brainstorming notes, or informal meeting summaries, unless those files contain structured data relevant for analysis. Keep personal or unplanned documents in OneDrive or SharePoint for collaboration only.

Medallion architecture layers

You should apply a medallion architecture to semi-structured and unstructured datasets. It includes content ingested from SharePoint and OneDrive. Unstructured files such as PDFs and Word documents typically lack a fixed schema. They become semi-structured when enriched with metadata or tags (for example, XML or JSON wrappers). The medallion architecture organizes data into three layers that progressively improve quality and usability:

1. **Bronze (raw ingestion):** Store all data in its original form in OneLake, including tabular files, JSON/XML, PDFs, and Office documents. Keep this layer immutable for audit and

lineage. Use shortcuts or mirroring for external sources to avoid duplication and maintain consistency.

2. **Silver (validated):** Clean and standardize data. Remove duplicates, normalize formats, and apply schemas. For example, convert PDFs to text or transform JSON into Delta tables. This layer supports accurate joins and indexing for retrieval.
3. **Gold (business context):** Aggregate data with business meaning, add semantic layers, optimize performance, and certify datasets. Register these curated data products in Microsoft Purview to enforce governance and compliance. Organizations should prioritize creating and certifying Gold tables for critical business domains such as finance, supply chain, and customer operations. This step ensures that AI agents deliver accurate insights and reduces the risk of inconsistent reporting.

To enable retrieval, create OneLake [search indexers](#) with Azure AI Search on Silver-layer data. Agents use Silver datasets to identify relationships and patterns. When an agent creates a curated set, store it as a registered data product in Purview or keep the instructions as a specification file in Git for version control. This process ensures reproducibility and governance. For an architecture, see [Fabric enterprise architecture](#).

Use Fabric data agents

Fabric Data Agents allow business teams to query **structured data** using natural language. These agents translate questions into precise SQL statements and return accurate results when they operate on high-quality, certified tables. They work best on mirrored or shortcut tables designated as Gold sources because these tables provide curated and governed data that supports consistent metrics and reliable analytics. Organizations should prioritize creating and certifying Gold tables for critical business domains such as finance, supply chain, and customer operations. This step ensures that AI agents deliver accurate insights and reduces the risk of inconsistent reporting.

Adaptive Gold (dynamic)

Adaptive Gold introduces a dynamic approach to data curation that responds to real-world usage patterns. Instead of relying on static indexing, organizations use agent-driven processes to create curated datasets that evolve with business needs.

Agents monitor frequent queries and aggregations to identify high-value patterns. They then materialize curated datasets as Delta tables or generate specification files to ensure reproducibility. Governance guardrails enforce approved metrics such as RevenuePerRegion or CustomerSatisfactionScore, and all dynamic datasets are registered in Microsoft Purview for compliance and visibility.

This adaptive model improves flexibility and performance because it prioritizes data that agents use most often. It also reduces latency for common queries and supports continuous optimization without manual intervention.

 Expand table

Aspect	Silver layer	Gold layer (static)	Adaptive gold (agent-driven)
Purpose	Clean, validated, normalized data	Business-ready, curated datasets with predefined metrics	Dynamic aggregations based on usage patterns and telemetry
Data quality	Deduplicated, standardized	Certified, governed, optimized for analytics	Optimized for frequent queries and evolving business needs
Structure	Normalized tables, semi-structured indexed	Semantic layer with KPIs, hierarchies, curated joins	Agent-generated tables or spec files for reproducibility
Performance	Good for ad-hoc queries	High performance (materialized views, preaggregations)	Improves over time as agents learn common queries
Governance	Registered in Purview	Certified and auditable in Purview	Registered dynamically, Enforced via Foundry + DSPM guardrails
Agent role	Consumes validated data	Queries curated sets for consistent metrics	Creates new curated sets or instructions for reuse
Flexibility	High (raw + normalized)	Low (static definitions)	Very high (adaptive to real-world usage)

Data retrieval strategies for custom agents

Organizations building AI agents must decide how the agent accesses data to answer questions and perform tasks. These choices directly affect accuracy, timeliness, and security. Agents generally combine two complementary approaches: agentic retrieval and Model Context Protocol (MCP) servers.

When an employee reports a VPN issue, an effective agent uses agentic retrieval to pull the troubleshooting guide from the organization’s approved knowledge sources. It checks the real-time VPN service status through an MCP read operation, and it can create a helpdesk ticket through an MCP write operation if the user requests it. Agentic retrieval provides curated, authoritative knowledge. MCP provides operational data and the ability to take actions.

Agentic retrieval and RAG: Agentic retrieval includes the retrieval-and-grounding aspects of traditional RAG, but extends them with reasoning, decomposition, parallel evidence gathering, and multi-source synthesis. It is the preferred term and recommended pattern for enterprise AI agents.

Both approaches work best when they draw from a unified, governed data foundation. Microsoft Fabric supports this through OneLake and the medallion architecture, organizing structured, semi-structured, and unstructured content. Adaptive Gold extends this model by allowing agents to optimize knowledge curation based on usage patterns.

See [agentic RAG](#) and [MCP servers](#).

Start with built-in retrieval options

Begin with the retrieval capabilities built into Microsoft Foundry and Copilot Studio. Built-in agentic retrieval and MCP features reduce integration complexity, accelerate deployment, and maintain governance consistency. Before investing in custom solutions, confirm whether these capabilities satisfy requirements for accuracy, compliance, and operational control. Examples of built-in capabilities include:

- [Foundry IQ](#)
- [Fabric IQ](#)
- [Fabric data agents](#)
- [Connect to MCP servers](#)
- [Sharepoint connection](#)
- [Azure AI Search](#)

Plan for agentic retrieval

When to use agentic retrieval: Use agentic retrieval when an agent must look up, explain, and cite information, especially from unstructured or semi-structured content such as policies, manuals, FAQs, product documentation, and curated extracts. Unlike a static RAG application that queries a single fixed index, agentic retrieval orchestrates a broader reasoning process. It decomposes user questions into sub-queries, runs parallel searches across approved sources, reconciles conflicting information, and synthesizes an answer grounded with citations. This provides just-in-time context and higher accuracy without locking the system into a static retrieval corpus or ranking strategy.

1. **Plan for a unified knowledge layer.** A unified knowledge layer consolidates retrievable content behind an approved endpoint or shared knowledge base so multiple agents and applications can produce consistent, grounded answers. These reusable knowledge bases typically center on domains such as employee policies, product documentation, and

support content. Foundry IQ is the preferred service for building this layer. It can incorporate curated Silver, Gold, and Adaptive Gold datasets produced by Fabric data agents, along with Azure AI Search indexes when authoritative answers are required. See [Foundry IQ](#).

2. **Plan for indexes.** Use an Azure AI Search index when you need tightly controlled retrieval behavior, including custom chunking, embeddings, fine-tuned ranking, or deterministic search over a defined corpus. Indexes complement, not replace, the unified knowledge layer.

Retrieval from Fabric OneLake: Create Azure AI Search OneLake Search [search indexers](#) over Silver-layer data. Silver is the correct layer for agent grounding because Silver retains the raw relationships, structure, and patterns. Gold datasets introduce aggregations and transformations that improve analytics but remove the detail agents need for reasoning, inference, and multi-source synthesis. Agents use Silver datasets to identify relationships and patterns, and may generate a curated dataset as a result. The curated dataset can be registered as a data product in Purview, or it can be stored as an instruction/specification file in Git so the agent can regenerate the curated set on demand. For architectural guidance, see [Fabric enterprise architecture](#).

Plan for building MCP servers

When to use MCP? Choose MCP when agents must take actions or access real-time data. For example, "How many units of SKU123 are in stock right now?" or "Create an incident ticket." MCP gives agents a standardized way to call tools hosted on remote servers, with identity, policy, and audit controls.

1. **Identify candidate systems.** Prioritize systems where live queries or transactions add value: CRM for pipeline updates, ERP for inventory and orders, ITSM for tickets, IoT telemetry for sensor readings, and internal APIs for specialized processes.
2. **Build or connect MCP servers.** Where there are available and built-in MCP servers in Foundry and Copilot Studio, use them. When there aren't, you need to build. Clarify the type of interaction the AI agent needs. Some systems require read-only access for queries, while others demand read-write capabilities to create or update records. This distinction influences security controls and compliance requirements. Define these roles early to avoid unnecessary risk and ensure governance aligns with operational needs. See [Connect to MCP servers](#) or [Build and register an MCP server](#).
3. **Enforce security and compliance for MCP.** Require authentication for every tool call. Foundry supports key-based access, Microsoft Entra Agent Identity, project managed

identity, and OAuth identity passthrough. It provides detailed auditing of tool invocations and data flows. Use RBAC on both the Foundry project and the target service, and prefer identity passthrough when user-level permissions must persist. See [Authentication support for MCP tool](#).

Document retrieval decisions

A successful data plan doesn't end with technical implementation. Leaders must document how AI agents access information so that governance, compliance, and accountability remain clear. Retrieval decisions affect accuracy, security, and operational risk. Without documentation, teams face ambiguity about which systems agents use and how those systems interact. Start by listing each major data domain the agent supports and classify the retrieval method for that domain. Use clear categories such as search, API, or both. For example:

- Product FAQs: Search using a knowledge index
- Order tracking: API call to the order management system
- Create return label: API call to the shipping system

This documentation provides transparency for stakeholders and developers. It also supports audits and compliance reviews by showing how data flows through the agent ecosystem. When retrieval strategies combine RAG and MCP, note which approach applies to each domain and why. For instance, RAG might handle static policy documents, while MCP manages real-time inventory queries. Include governance details in the same record. Specify which data sources are certified, which APIs require authentication, and which permissions apply. This step ensures that retrieval aligns with organizational security policies and regulatory requirements.

Next step

Responsible AI

Last updated on 12/16/2025

Establishing responsible AI policies for AI agents across organizations

This article explains how to establish responsible AI policies that ensure ethical, transparent, and accountable AI agent deployment across your organization. You learn how to create practical governance standards that integrate into existing workflows rather than creating bureaucratic obstacles. Responsible AI policies form the foundation of the **Govern agents** step in the AI agent adoption process (see *figure 1*).



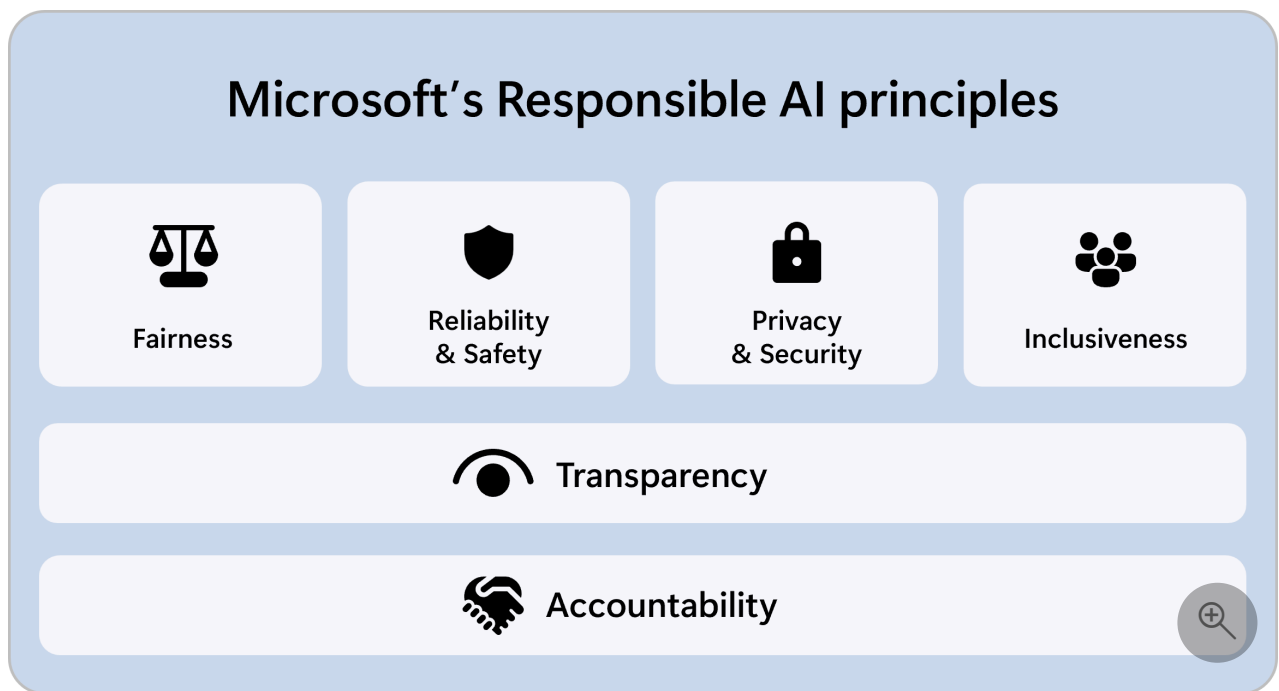
Figure 1. Microsoft's AI agent adoption process.

Without clear responsible AI policies, organizations face significant risks from AI agents. These risks include reputational damage from biased or harmful AI outputs, regulatory penalties from noncompliance with emerging AI laws, and erosion of stakeholder trust that undermines AI adoption efforts.

Define responsible AI standards

Your responsible AI standards establish the baseline for all AI agents in your organization. These standards translate abstract principles into concrete requirements that development teams can implement. The most effective standards integrate seamlessly with existing governance processes rather than creating parallel compliance tracks.

1. **Adopt industry frameworks as your baseline.** Start with established frameworks like [Microsoft's Responsible AI principles](#) or [NIST's AI Risk Management Framework](#). These frameworks provide comprehensive coverage of fairness, privacy, transparency, and accountability considerations. Using proven frameworks accelerates policy development and ensures you address critical areas. Industry frameworks also provide credibility when communicating with stakeholders, regulators, and partners who recognize these standards.



2. **Align AI policies with corporate governance.** Map your responsible AI requirements to existing corporate policies for data governance, security, and risk management. This alignment prevents AI from becoming an exception to normal oversight procedures. When AI policies integrate with established processes, teams already familiar with corporate governance naturally extend their practices to AI systems. Review your policies against emerging regulations like the EU AI Act and ISO/IEC 42001 standards quarterly to maintain compliance as the regulatory landscape evolves.
3. **Implement "ethical by design" practices.** Provide development teams with practical tools including templates for ethical impact assessments, checklists for bias testing, and code libraries for transparency features. These resources transform abstract principles into concrete actions. Establish mandatory training programs that explain both the *what* and *why* of responsible AI requirements. Create an AI champion network across business units where experienced practitioners guide their peers through ethical considerations specific to their domains.

Assign ownership and oversight

Clear ownership structures ensure consistent policy enforcement while empowering teams to innovate within defined boundaries. The most successful organizations create centralized standards with distributed implementation, avoiding both chaos and bureaucracy.

1. **Empower a cross-functional governance team.** Establish an [AI Center of Excellence](#) or ethics committee that includes representatives from legal, security, product, and engineering teams. This group defines standards and provides consultative support rather than acting as gatekeepers. Give this team executive sponsorship and clear authority to enforce policies when necessary. The governance team should develop standard

operating procedures, review high-risk projects, and evolve policies based on lessons learned. Resource this team adequately to prevent it from becoming a bottleneck that slows innovation.

2. **Institutionalize governance in workflows.** Embed responsible AI checkpoints at key project milestones including design reviews, testing phases, and prelaunch approvals. Require formal sign-offs from the governance team for high-risk AI agents that interact directly with customers or make consequential decisions. Implement automated scanning tools that detect potentially biased training data, inappropriate content generation, or privacy violations. These automated checks provide continuous compliance monitoring without relying solely on manual reviews.

Establish auditing and transparency

Ongoing auditing and transparency mechanisms ensure AI agents continue meeting ethical standards after deployment. These processes build trust with users and provide accountability when issues arise.

1. **Set up auditing and incident response procedures.** Schedule regular audits of deployed AI agents to verify continued compliance with responsible AI standards. Monitor for model drift, emergent biases, or changing risk profiles as systems learn from new data. Establish clear incident response protocols that define escalation paths, shutdown authorities, and communication procedures. Document who can make decisions about taking an AI agent offline, how to notify affected users, and what remediation steps to follow. Practice these procedures through tabletop exercises before real incidents occur.
2. **Maintain transparency and stakeholder communication.** Document all governance decisions including risk assessments, mitigation strategies, and approval rationales. This documentation supports regulatory compliance, enables knowledge transfer, and facilitates continuous improvement. Publish your organization's commitment to responsible AI principles publicly. Ensure every AI agent clearly identifies itself as artificial intelligence to users. Provide accessible feedback mechanisms where users can report concerns or challenge AI-generated decisions. Regular transparency reports that detail AI usage, incident statistics, and improvement initiatives build stakeholder confidence.

Next steps

Transform these responsible AI policies into actionable governance frameworks that support secure and scalable AI agent adoption across your organization.

Govern agents across organization

Last updated on 12/03/2025

Governance and security for AI agents across the organization

This article explains how to establish governance and security practices for AI agents across your organization. The formation of AI agent governance policies is critical to the **Govern agents** step in the AI agent adoption process (see figure 1).

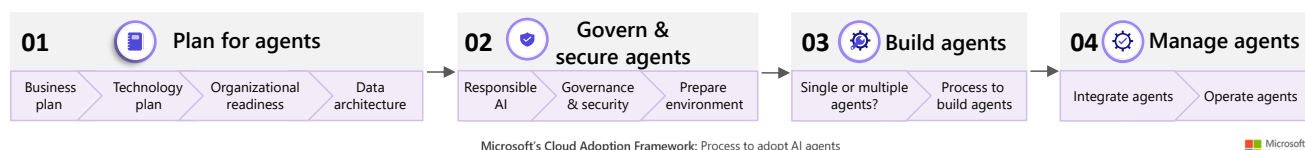
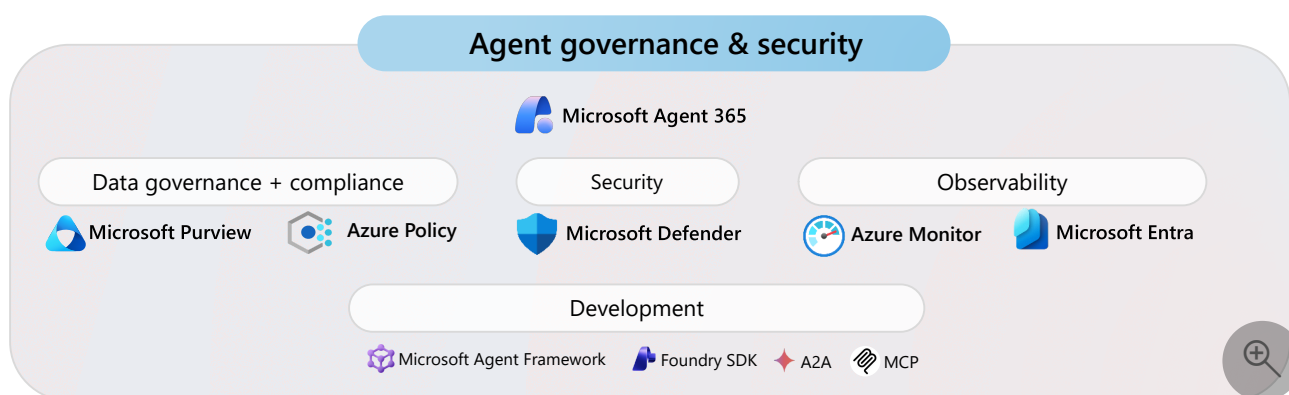


Figure 1. Microsoft's AI agent adoption process.

Without proper governance, AI agents can introduce risks related to sensitive data exposure, compliance boundaries, and security vulnerabilities. This guidance shows how to implement controls that help protect data, support regulatory compliance, maintain visibility into agent behavior, and secure agent infrastructure throughout its lifecycle. Organizations that establish these controls are better positioned to avoid regulatory issues, data breaches, and operational disruptions associated with ungoverned agent behaviors.



Data governance and compliance

Organizations require concrete mechanisms to control how agents access, process, and store data. These mechanisms translate regulatory requirements and corporate policies into technical controls that enforce boundaries around agent behavior. Data governance establishes the foundation for responsible AI deployment by defining what data agents can use, where they can operate, and how long they can retain information.

Regulatory compliance

All agents must comply with regulations and standards. Regulatory compliance encompasses data protection laws (such as GDPR and HIPAA), industry certifications, and internal governance requirements. Translate these regulations into foundational controls to ensure agents process data responsibly, securely, and transparently.

1. **Enforce data privacy.** Require agents to follow data privacy principles and use only the data necessary for intended functionality. Review datasets fed into AI models for RAG, fine-tuning, or training to identify privacy risks. Audit memory stores and logs regularly. Anonymize or pseudonymize personal data where feasible. Support user rights, such as deletion requests, to align with organizational privacy policies.
2. **Mandate data residency compliance.** Ensure agents operate in environments that meet data residency and sovereignty requirements. Identify the location of each data source, agent runtime, and output storage to ensure compliance. Use Azure regions or on-premises environments that align with residency policies. Configure logging to avoid unnecessary storage of sensitive content, retaining only what compliance or operational policies require, and ensure data remains encrypted at rest.
3. **Define data retention policies.** Enforce defined retention periods for logs, memory, and training data. Implement automated purging or anonymization processes to retain only the context necessary for agent functionality. Require agents to inform users about retention durations and provide deletion mechanisms. Extend retention policies into full lifecycle management, covering data creation, archival, deletion, and purging.

Microsoft facilitation:

 **Foundry:** Use [Microsoft Purview Compliance Manager](#) to translate regulations, such as the EU AI Act, into controls and assess compliance posture across AI applications. Use [Microsoft Purview APIs](#) to integrate compliance automation into agent workflows. Review [Microsoft Purview capabilities for Foundry](#) to understand data governance and protection options.

 **Copilot Studio:** Follow [governance and security best practices](#). Use [data locations](#) to respect data sovereignty. See the platform's [compliance \(ISO, SOC, HIPAA\) certifications](#).

Corporate compliance

All agents must comply with Responsible AI policies. Corporate compliance ensures agents align with internal governance policies grounded in Responsible AI principles: fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability. Organizations translate these principles into concrete controls that govern how agents handle data, make decisions, and interact with users. These controls protect the organization from reputational harm, regulatory penalties, and loss of user trust while ensuring agents operate within defined ethical and operational boundaries.

1. **Isolate confidential data.** Separate confidential data from public data sources using physical or logical boundaries. Public-facing agents must not access internal business data. This separation prevents confidential information from unintentionally reaching public channels. In an Azure landing zone, deploy internal agents and data in "corp" (internal) management group subscriptions and public agents and data in "online" management group subscriptions.
2. **Restrict data access and enforce permissions.** Grant agents access only to the specific data sources required for their function. Don't provide broad access to all organizational data. When an agent accesses data on behalf of a user, ensure it inherits that user's permissions. Pass the user's identity or token securely when querying data to ensure session integrity. Apply data loss prevention (DLP) policies and sensitivity labels to protect data. This approach ensures that an internal helpdesk agent, for example, shows an employee only their own HR record.
3. **Standardize knowledge and tool integrations.** Standardize how agents connect to knowledge and tools to reduce duplication and simplify maintenance. Use official APIs and connectors instead of custom integrations. It allows multiple agents across departments to reuse infrastructure and reduces operational costs. Follow governance policies that use project-level data isolation to prevent cross-contamination.
4. **Mandate transparency.** Disclose AI involvement clearly in agent interfaces. Notify internal stakeholders when their data supports training or inference. Review new data sources before integration to validate content, permissions, and security risks.
5. **Enforce fairness, inclusiveness, and accountability.** Assign clear ownership for AI outcomes to ensure alignment with organizational Responsible AI policies. Use Responsible AI tools to evaluate models for bias and inclusiveness during development and testing phases. Review model evaluations regularly to verify that agents treat all user groups fairly. Designate a human role accountable for approving deployments and monitoring ongoing compliance with ethical standards.

Microsoft facilitation:



Foundry: Review [Microsoft Purview capabilities for Foundry](#) to understand data governance and protection options. Use [Manage agent identities](#) to control access. Use [Azure Policy](#) to control infrastructure configurations and [model deployment](#). [Create custom policies](#) as needed.



Copilot Studio: [Create and manage solution pipelines](#) and use [reusable component collections](#) and enforcing version governance.

Agent observability

The actions of every agent must be auditable. Observability enables you to monitor and understand what AI agents do both in real time and over time. Because agents operate probabilistically and don't always behave the same way, visibility into their actions becomes essential for managing risk, ensuring compliance, and optimizing performance. Organizations require complete visibility into agent deployments, behaviors, and costs to maintain control at scale. Consider [Microsoft Agent 365](#) as the main tool for agent observability across your organization.

1. **Assign unique identities.** Every AI agent requires a unique identity that includes ownership details, version history, and lifecycle status. Use [Microsoft Entra Agent Identity](#) to establish these identities, which helps the organization distinguish between production, development, and test agents, providing accountability and risk management.
2. **Maintain agent inventory.** Untracked or "shadow" deployments pose security and cost risks. Organizations discover and classify all AI workloads across the cloud environment to maintain a complete inventory of AI assets.
3. **Centralize logging.** Track key logs consistently to a centralized location, such as an Azure Log Analytics workspace. This consistency improves transparency, supports audit readiness, and simplifies troubleshooting. Centralized logs enable cross-team collaboration. Gather custom telemetry that captures how agents behave and how users interact with them. This information decides whether to scale, refine, or retire agents based on business impact. Build dashboards that provide leadership with a clear view of how AI investments contribute to outcomes.
4. **Track and allocate costs.** AI agents consume resources such as compute power, tokens, and API calls. Without visibility, costs escalate quickly. Establish a unified view of agent usage and cost across departments and projects to track metrics like token consumption and compute usage. Organize this data by department or project to identify where costs concentrate. Apply cost center tags to allocate agent expenses accurately. Require teams to tag resources per agent or use case to visualize cost breakdowns clearly. Set up real-time alerts to notify teams when spending approaches budget thresholds. These alerts prevent overruns and support proactive financial management. Restrict who can create, deploy, and scale agents to reduce risk and ensure only authorized personnel manage AI deployments.

Microsoft facilitation:



Foundry: Manage agent identities with [Microsoft Entra Agent Identity](#). Publish agents to [Agent 365](#) for central observability. [Monitor agents](#), [monitor model deployments](#), [monitor applications](#) using [dashboards](#). [Plan and manage costs](#), and use the [management center](#) to

centrally administer quotas and access. If Agent 365 is unavailable, use [Microsoft Defender for Cloud](#) to discover and categorize agent workloads.

 **Copilot Studio:** [Monitor logging and auditing](#), centralize data with [Azure Application Insights in Azure Monitor](#), and [review usage and message allocation](#) to manage consumption.

Agent security

All agents must meet all baseline security requirements. AI agents process natural language, interact with external sources, and often make autonomous decisions. These capabilities introduce security risks that organizations must address, including data leakage, data poisoning, jailbreak attempts, and credential theft. Organizations integrate agent security into existing enterprise security frameworks to ensure consistency, reduce exposure, and support regulatory compliance.

1. **Mandate agent security training.** Provide targeted training for security and development teams. Focus on real-world scenarios of jailbreak techniques that bypass agent safeguards, data poisoning attacks that manipulate training data, and credential theft through agent interfaces. This training builds awareness and readiness, enabling teams to respond effectively to AI-specific threats.
2. **Require AI threat protection.** Activate AI-specific threat protection. Microsoft Defender for Cloud's [AI threat protection](#) detects prompt manipulation, unauthorized data access, and other agent-specific threats. These protections use global threat intelligence and integrate with Azure AI Content Safety to identify suspicious behavior in real time. Activating these tools reduces exposure and enables faster response to emerging risks.
3. **Enforce infrastructure security baselines.** Treat agent infrastructure with the same level of oversight as customer-facing systems. Agent runtimes require the same governance and oversight as customer-facing systems to prevent unauthorized access and ensure stability.
4. **Mandate adversarial testing.** Validate agent resilience before deployment. Red teaming simulates real-world attacks to identify vulnerabilities that standard testing might miss. These exercises detect weaknesses such as prompt injection, data leakage, and jailbreak attempts. Run adversarial tests on all agents before production release and after significant updates. This step reduces the likelihood of exploitation and improves the agent's ability to handle hostile input.
5. **Filter inputs and outputs.** Block adversarial input that attempts to bypass integrated safety protocols. Treat all incoming data, text, files, and images as potentially hostile. Validate and filter inputs before they reach models or backend systems. Strip scripting

and injection content from text, enforce type and size restrictions for files, and scan media using moderation services. Update sanitization rules based on observed attack patterns.

6. **Standardize authentication.** Mandate the use of [managed identities for authentication](#) to eliminate credential management risks. This approach removes the need for developers to handle secrets and ensures that only authorized identities can access resources.
7. **Enforce least privilege.** Govern agent capabilities tightly when they execute actions. Each tool the agent uses enforces the user's permissions, or it has scoped service accounts. Use Data Loss Prevention (DLP) policies to restrict what data the agent can access or output. For instance, prevent the agent from returning credit card numbers in answers.
8. **Integrate with security operations.** Ensure AI-related alerts flow into the Security Operations Center (SOC). Define thresholds for anomalies such as latency spikes or unauthorized access attempts. Use Azure Monitor Alerts and route them to Microsoft Sentinel through Log Analytics. This integration strengthens the organization's overall security posture and ensures agents remain resilient and trustworthy.
9. **Govern external integrations.** Support interoperability standards such as MCP and emerging standards like A2A when agents interact with external systems. Restrict external interactions to trusted MCP servers that meet organizational security and compliance standards. Validate all external agent communications and limit them to explicitly approved entities. Define clear boundaries for data access and logic execution during external interactions. These measures prevent leakage of sensitive information and maintain control over agent behavior across organizational boundaries.
10. **Establish incident response plans.** Decide in advance how to quickly disable an agent if it malfunctions or causes harm. Plan how to communicate incidents. Include steps for preserving logs for forensic analysis and updating stakeholders. Establish disaster recovery plans for AI agents as you do for systems. Run drills if the agent supports critical operations to ensure the team knows how to respond under pressure.

Microsoft facilitation:



Foundry: Review the [Azure security baseline for Foundry](#) as a standard policy. Enforce the use of the [AI Red Teaming Agent](#) to scan applications for safety and security issues before deployment. Also see [AI Red Teaming Playground Labs](#) [↗](#). Enforce baseline [guardrails and controls](#) to mitigate harmful content. Combine with [Purview DLP](#) policies or add custom detectors, either within Purview or as standalone solutions, to reduce sensitive data leakage. Enforce least-privilege roles using [Azure role-based access control](#). Enforce the [high availability and resiliency](#) guidance across your organization.



Copilot Studio: Review [security and governance guidance](#), enable [automatic security scans](#), and verify [agent runtime protection status](#) to ensure continuous validation.

Agent development

Every agent must only use approved frameworks and protocols. Organizations standardize how agents interact with tools, data, and each other to ensure consistency, safety, and interoperability. Business leaders define which protocols and frameworks are acceptable and mandate their use across all agent development efforts.

1. **Standardize agent frameworks.** Establish a consistent set of frameworks and SDKs for agent development. This approach reduces complexity, improves collaboration across departments, and simplifies long-term maintenance. Standardize on options that provide structured approaches to agent orchestration and integration and support rapid onboarding of new agents. When selecting frameworks, ensure compatibility with existing infrastructure so the framework integrates well with current cloud architecture and data sources. Choose frameworks with robust documentation and active support to reduce ramp-up time. Favor frameworks that include built-in compliance and security features to meet internal policy requirements. Standardizing these choices creates a predictable development environment that accelerates delivery and reduces risk.
2. **Standardize agent protocols.** Protocols define how agents interact with tools, data, and each other. All agent development efforts use standard protocols to ensure secure and predictable behavior. Two key protocols to adopt are the Model Context Protocol (MCP) and the Agent-to-Agent Protocol (A2A). MCP provides structured and secure access to tools, APIs, and data sources. It enforces boundaries around what agents can access, which prevents unauthorized actions and supports compliance. A2A enables consistent communication between agents. It supports task delegation and context sharing, which improves coordination and reduces errors in multi-agent systems. These protocols create a common language for agents across the organization. They reduce integration overhead, improve interoperability, and support governance by enforcing clear interaction rules.

Microsoft facilitation:



Foundry: Standardize development with the [Microsoft Agent Framework](#) and the [Foundry SDK](#). These tools provide structured approaches to agent orchestration and integration.



Copilot Studio: Use the [authoring canvas](#) for low-code development and [knowledge sources](#) to integrate with external data and services.

Prepare environment

An environment is the controlled space where workloads and AI agents are developed, deployed, and operated. The goal is to ensure that every AI environment enforces governance

policies that act as guardrails, delivering consistency, security, and compliance across all workloads and agents. These guardrails protect teams while enabling rapid, safe development.

1. **Collaborate on enforcement strategy.** Governance must balance control with agility. Overly rigid policies can slow development and stifle innovation. Partner with workload teams to define an approach that starts with monitoring rather than immediate restrictions. Begin with an audit-based model to observe behaviors and identify patterns, then gradually introduce stricter controls as needed. This phased approach minimizes disruption while refining guardrails over time.
2. **Govern the environments.** Governing environments means applying consistent organizational policies and controls across all platforms, whether PaaS, IaaS, or SaaS. Organizations are responsible for ensuring every environment aligns with corporate standards and regulatory requirements.
 - **PaaS and IaaS platforms:** Establish consistently governed application landing zones for building agents. These application landing zones provide a foundation for identity, networking, and security controls. AI agents do not require a specialized application landing zone architecture, but governance policies should reflect the services in use. For detailed guidance, see [Ready your AI environment](#).
 - **SaaS platforms:** A landing zone foundation is not required because the platform is fully managed. Instead, focus on configuring identity, access, and data governance settings in line with organizational policies and best practices. Apply compliance controls wherever possible to maintain security and regulatory alignment.

Microsoft facilitation:



Foundry: For infrastructure patterns, start with the [Foundry chat reference architecture](#) and/or the [AI application landing zone](#) and modify to fit your needs. Review the [AI workloads](#) guidance in the Well-Architected Framework.



Copilot Studio: Follow [governance and security best practices](#). [Configure environments](#) to separate workloads. Apply [Data Loss Prevention \(DLP\) policies](#) to control connector usage within each environment. [Create agents from templates](#).

Next step

Single agent or multiple agents?

Governance and security for AI agents across the organization

This article explains how to establish governance and security practices for AI agents across your organization. The formation of AI agent governance policies is critical to the **Govern agents** step in the AI agent adoption process (see figure 1).



Figure 1. Microsoft's AI agent adoption process.

Without proper governance, AI agents can introduce risks related to sensitive data exposure, compliance boundaries, and security vulnerabilities. This guidance shows how to implement controls that help protect data, support regulatory compliance, maintain visibility into agent behavior, and secure agent infrastructure throughout its lifecycle. Organizations that establish these controls are better positioned to avoid regulatory issues, data breaches, and operational disruptions associated with ungoverned agent behaviors.



Data governance and compliance

Organizations require concrete mechanisms to control how agents access, process, and store data. These mechanisms translate regulatory requirements and corporate policies into technical controls that enforce boundaries around agent behavior. Data governance establishes the foundation for responsible AI deployment by defining what data agents can use, where they can operate, and how long they can retain information.

Regulatory compliance

All agents must comply with regulations and standards. Regulatory compliance encompasses data protection laws (such as GDPR and HIPAA), industry certifications, and internal governance requirements. Translate these regulations into foundational controls to ensure agents process data responsibly, securely, and transparently.

1. **Enforce data privacy.** Require agents to follow data privacy principles and use only the data necessary for intended functionality. Review datasets fed into AI models for RAG, fine-tuning, or training to identify privacy risks. Audit memory stores and logs regularly. Anonymize or pseudonymize personal data where feasible. Support user rights, such as deletion requests, to align with organizational privacy policies.
2. **Mandate data residency compliance.** Ensure agents operate in environments that meet data residency and sovereignty requirements. Identify the location of each data source, agent runtime, and output storage to ensure compliance. Use Azure regions or on-premises environments that align with residency policies. Configure logging to avoid unnecessary storage of sensitive content, retaining only what compliance or operational policies require, and ensure data remains encrypted at rest.
3. **Define data retention policies.** Enforce defined retention periods for logs, memory, and training data. Implement automated purging or anonymization processes to retain only the context necessary for agent functionality. Require agents to inform users about retention durations and provide deletion mechanisms. Extend retention policies into full lifecycle management, covering data creation, archival, deletion, and purging.

Microsoft facilitation:

 **Foundry:** Use [Microsoft Purview Compliance Manager](#) to translate regulations, such as the EU AI Act, into controls and assess compliance posture across AI applications. Use [Microsoft Purview APIs](#) to integrate compliance automation into agent workflows. Review [Microsoft Purview capabilities for Foundry](#) to understand data governance and protection options.

 **Copilot Studio:** Follow [governance and security best practices](#). Use [data locations](#) to respect data sovereignty. See the platform's [compliance \(ISO, SOC, HIPAA\) certifications](#).

Corporate compliance

All agents must comply with Responsible AI policies. Corporate compliance ensures agents align with internal governance policies grounded in Responsible AI principles: fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability. Organizations translate these principles into concrete controls that govern how agents handle data, make decisions, and interact with users. These controls protect the organization from reputational harm, regulatory penalties, and loss of user trust while ensuring agents operate within defined ethical and operational boundaries.

1. **Isolate confidential data.** Separate confidential data from public data sources using physical or logical boundaries. Public-facing agents must not access internal business data. This separation prevents confidential information from unintentionally reaching public channels. In an Azure landing zone, deploy internal agents and data in "corp" (internal) management group subscriptions and public agents and data in "online" management group subscriptions.
2. **Restrict data access and enforce permissions.** Grant agents access only to the specific data sources required for their function. Don't provide broad access to all organizational data. When an agent accesses data on behalf of a user, ensure it inherits that user's permissions. Pass the user's identity or token securely when querying data to ensure session integrity. Apply data loss prevention (DLP) policies and sensitivity labels to protect data. This approach ensures that an internal helpdesk agent, for example, shows an employee only their own HR record.
3. **Standardize knowledge and tool integrations.** Standardize how agents connect to knowledge and tools to reduce duplication and simplify maintenance. Use official APIs and connectors instead of custom integrations. It allows multiple agents across departments to reuse infrastructure and reduces operational costs. Follow governance policies that use project-level data isolation to prevent cross-contamination.
4. **Mandate transparency.** Disclose AI involvement clearly in agent interfaces. Notify internal stakeholders when their data supports training or inference. Review new data sources before integration to validate content, permissions, and security risks.
5. **Enforce fairness, inclusiveness, and accountability.** Assign clear ownership for AI outcomes to ensure alignment with organizational Responsible AI policies. Use Responsible AI tools to evaluate models for bias and inclusiveness during development and testing phases. Review model evaluations regularly to verify that agents treat all user groups fairly. Designate a human role accountable for approving deployments and monitoring ongoing compliance with ethical standards.

Microsoft facilitation:



Foundry: Review [Microsoft Purview capabilities for Foundry](#) to understand data governance and protection options. Use [Manage agent identities](#) to control access. Use [Azure Policy](#) to control infrastructure configurations and [model deployment](#). [Create custom policies](#) as needed.



Copilot Studio: [Create and manage solution pipelines](#) and use [reusable component collections](#) and enforcing version governance.

Agent observability

The actions of every agent must be auditable. Observability enables you to monitor and understand what AI agents do both in real time and over time. Because agents operate probabilistically and don't always behave the same way, visibility into their actions becomes essential for managing risk, ensuring compliance, and optimizing performance. Organizations require complete visibility into agent deployments, behaviors, and costs to maintain control at scale. Consider [Microsoft Agent 365](#) as the main tool for agent observability across your organization.

1. **Assign unique identities.** Every AI agent requires a unique identity that includes ownership details, version history, and lifecycle status. Use [Microsoft Entra Agent Identity](#) to establish these identities, which helps the organization distinguish between production, development, and test agents, providing accountability and risk management.
2. **Maintain agent inventory.** Untracked or "shadow" deployments pose security and cost risks. Organizations discover and classify all AI workloads across the cloud environment to maintain a complete inventory of AI assets.
3. **Centralize logging.** Track key logs consistently to a centralized location, such as an Azure Log Analytics workspace. This consistency improves transparency, supports audit readiness, and simplifies troubleshooting. Centralized logs enable cross-team collaboration. Gather custom telemetry that captures how agents behave and how users interact with them. This information decides whether to scale, refine, or retire agents based on business impact. Build dashboards that provide leadership with a clear view of how AI investments contribute to outcomes.
4. **Track and allocate costs.** AI agents consume resources such as compute power, tokens, and API calls. Without visibility, costs escalate quickly. Establish a unified view of agent usage and cost across departments and projects to track metrics like token consumption and compute usage. Organize this data by department or project to identify where costs concentrate. Apply cost center tags to allocate agent expenses accurately. Require teams to tag resources per agent or use case to visualize cost breakdowns clearly. Set up real-time alerts to notify teams when spending approaches budget thresholds. These alerts prevent overruns and support proactive financial management. Restrict who can create, deploy, and scale agents to reduce risk and ensure only authorized personnel manage AI deployments.

Microsoft facilitation:



Foundry: Manage agent identities with [Microsoft Entra Agent Identity](#). Publish agents to [Agent 365](#) for central observability. [Monitor agents](#), [monitor model deployments](#), [monitor applications](#) using [dashboards](#). [Plan and manage costs](#), and use the [management center](#) to

centrally administer quotas and access. If Agent 365 is unavailable, use [Microsoft Defender for Cloud](#) to discover and categorize agent workloads.

 **Copilot Studio:** [Monitor logging and auditing](#), centralize data with [Azure Application Insights in Azure Monitor](#), and [review usage and message allocation](#) to manage consumption.

Agent security

All agents must meet all baseline security requirements. AI agents process natural language, interact with external sources, and often make autonomous decisions. These capabilities introduce security risks that organizations must address, including data leakage, data poisoning, jailbreak attempts, and credential theft. Organizations integrate agent security into existing enterprise security frameworks to ensure consistency, reduce exposure, and support regulatory compliance.

1. **Mandate agent security training.** Provide targeted training for security and development teams. Focus on real-world scenarios of jailbreak techniques that bypass agent safeguards, data poisoning attacks that manipulate training data, and credential theft through agent interfaces. This training builds awareness and readiness, enabling teams to respond effectively to AI-specific threats.
2. **Require AI threat protection.** Activate AI-specific threat protection. Microsoft Defender for Cloud's [AI threat protection](#) detects prompt manipulation, unauthorized data access, and other agent-specific threats. These protections use global threat intelligence and integrate with Azure AI Content Safety to identify suspicious behavior in real time. Activating these tools reduces exposure and enables faster response to emerging risks.
3. **Enforce infrastructure security baselines.** Treat agent infrastructure with the same level of oversight as customer-facing systems. Agent runtimes require the same governance and oversight as customer-facing systems to prevent unauthorized access and ensure stability.
4. **Mandate adversarial testing.** Validate agent resilience before deployment. Red teaming simulates real-world attacks to identify vulnerabilities that standard testing might miss. These exercises detect weaknesses such as prompt injection, data leakage, and jailbreak attempts. Run adversarial tests on all agents before production release and after significant updates. This step reduces the likelihood of exploitation and improves the agent's ability to handle hostile input.
5. **Filter inputs and outputs.** Block adversarial input that attempts to bypass integrated safety protocols. Treat all incoming data, text, files, and images as potentially hostile. Validate and filter inputs before they reach models or backend systems. Strip scripting

and injection content from text, enforce type and size restrictions for files, and scan media using moderation services. Update sanitization rules based on observed attack patterns.

6. **Standardize authentication.** Mandate the use of [managed identities for authentication](#) to eliminate credential management risks. This approach removes the need for developers to handle secrets and ensures that only authorized identities can access resources.
7. **Enforce least privilege.** Govern agent capabilities tightly when they execute actions. Each tool the agent uses enforces the user's permissions, or it has scoped service accounts. Use Data Loss Prevention (DLP) policies to restrict what data the agent can access or output. For instance, prevent the agent from returning credit card numbers in answers.
8. **Integrate with security operations.** Ensure AI-related alerts flow into the Security Operations Center (SOC). Define thresholds for anomalies such as latency spikes or unauthorized access attempts. Use Azure Monitor Alerts and route them to Microsoft Sentinel through Log Analytics. This integration strengthens the organization's overall security posture and ensures agents remain resilient and trustworthy.
9. **Govern external integrations.** Support interoperability standards such as MCP and emerging standards like A2A when agents interact with external systems. Restrict external interactions to trusted MCP servers that meet organizational security and compliance standards. Validate all external agent communications and limit them to explicitly approved entities. Define clear boundaries for data access and logic execution during external interactions. These measures prevent leakage of sensitive information and maintain control over agent behavior across organizational boundaries.
10. **Establish incident response plans.** Decide in advance how to quickly disable an agent if it malfunctions or causes harm. Plan how to communicate incidents. Include steps for preserving logs for forensic analysis and updating stakeholders. Establish disaster recovery plans for AI agents as you do for systems. Run drills if the agent supports critical operations to ensure the team knows how to respond under pressure.

Microsoft facilitation:



Foundry: Review the [Azure security baseline for Foundry](#) as a standard policy. Enforce the use of the [AI Red Teaming Agent](#) to scan applications for safety and security issues before deployment. Also see [AI Red Teaming Playground Labs](#) [↗](#). Enforce baseline [guardrails and controls](#) to mitigate harmful content. Combine with [Purview DLP](#) policies or add custom detectors, either within Purview or as standalone solutions, to reduce sensitive data leakage. Enforce least-privilege roles using [Azure role-based access control](#). Enforce the [high availability and resiliency](#) guidance across your organization.



Copilot Studio: Review [security and governance guidance](#), enable [automatic security scans](#), and verify [agent runtime protection status](#) to ensure continuous validation.

Agent development

Every agent must only use approved frameworks and protocols. Organizations standardize how agents interact with tools, data, and each other to ensure consistency, safety, and interoperability. Business leaders define which protocols and frameworks are acceptable and mandate their use across all agent development efforts.

1. **Standardize agent frameworks.** Establish a consistent set of frameworks and SDKs for agent development. This approach reduces complexity, improves collaboration across departments, and simplifies long-term maintenance. Standardize on options that provide structured approaches to agent orchestration and integration and support rapid onboarding of new agents. When selecting frameworks, ensure compatibility with existing infrastructure so the framework integrates well with current cloud architecture and data sources. Choose frameworks with robust documentation and active support to reduce ramp-up time. Favor frameworks that include built-in compliance and security features to meet internal policy requirements. Standardizing these choices creates a predictable development environment that accelerates delivery and reduces risk.
2. **Standardize agent protocols.** Protocols define how agents interact with tools, data, and each other. All agent development efforts use standard protocols to ensure secure and predictable behavior. Two key protocols to adopt are the Model Context Protocol (MCP) and the Agent-to-Agent Protocol (A2A). MCP provides structured and secure access to tools, APIs, and data sources. It enforces boundaries around what agents can access, which prevents unauthorized actions and supports compliance. A2A enables consistent communication between agents. It supports task delegation and context sharing, which improves coordination and reduces errors in multi-agent systems. These protocols create a common language for agents across the organization. They reduce integration overhead, improve interoperability, and support governance by enforcing clear interaction rules.

Microsoft facilitation:



Foundry: Standardize development with the [Microsoft Agent Framework](#) and the [Foundry SDK](#). These tools provide structured approaches to agent orchestration and integration.



Copilot Studio: Use the [authoring canvas](#) for low-code development and [knowledge sources](#) to integrate with external data and services.

Prepare environment

An environment is the controlled space where workloads and AI agents are developed, deployed, and operated. The goal is to ensure that every AI environment enforces governance

policies that act as guardrails, delivering consistency, security, and compliance across all workloads and agents. These guardrails protect teams while enabling rapid, safe development.

1. **Collaborate on enforcement strategy.** Governance must balance control with agility. Overly rigid policies can slow development and stifle innovation. Partner with workload teams to define an approach that starts with monitoring rather than immediate restrictions. Begin with an audit-based model to observe behaviors and identify patterns, then gradually introduce stricter controls as needed. This phased approach minimizes disruption while refining guardrails over time.
2. **Govern the environments.** Governing environments means applying consistent organizational policies and controls across all platforms, whether PaaS, IaaS, or SaaS. Organizations are responsible for ensuring every environment aligns with corporate standards and regulatory requirements.
 - **PaaS and IaaS platforms:** Establish consistently governed application landing zones for building agents. These application landing zones provide a foundation for identity, networking, and security controls. AI agents do not require a specialized application landing zone architecture, but governance policies should reflect the services in use. For detailed guidance, see [Ready your AI environment](#).
 - **SaaS platforms:** A landing zone foundation is not required because the platform is fully managed. Instead, focus on configuring identity, access, and data governance settings in line with organizational policies and best practices. Apply compliance controls wherever possible to maintain security and regulatory alignment.

Microsoft facilitation:



Foundry: For infrastructure patterns, start with the [Foundry chat reference architecture](#) and/or the [AI application landing zone](#) and modify to fit your needs. Review the [AI workloads](#) guidance in the Well-Architected Framework.



Copilot Studio: Follow [governance and security best practices](#). [Configure environments](#) to separate workloads. Apply [Data Loss Prevention \(DLP\) policies](#) to control connector usage within each environment. [Create agents from templates](#).

Next step

Single agent or multiple agents?

Single agent or multiple agents

This article provides criteria to help you decide whether to build single-agent or multi-agent systems across your organization. Organizations face this decision during the **Build agents** phase of AI agent adoption (see *figure 1*).

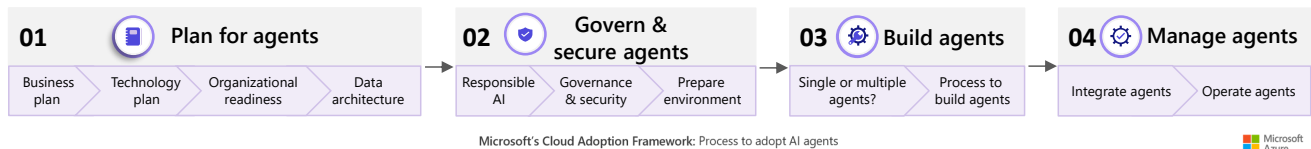


Figure 1. Microsoft's AI agent adoption process.

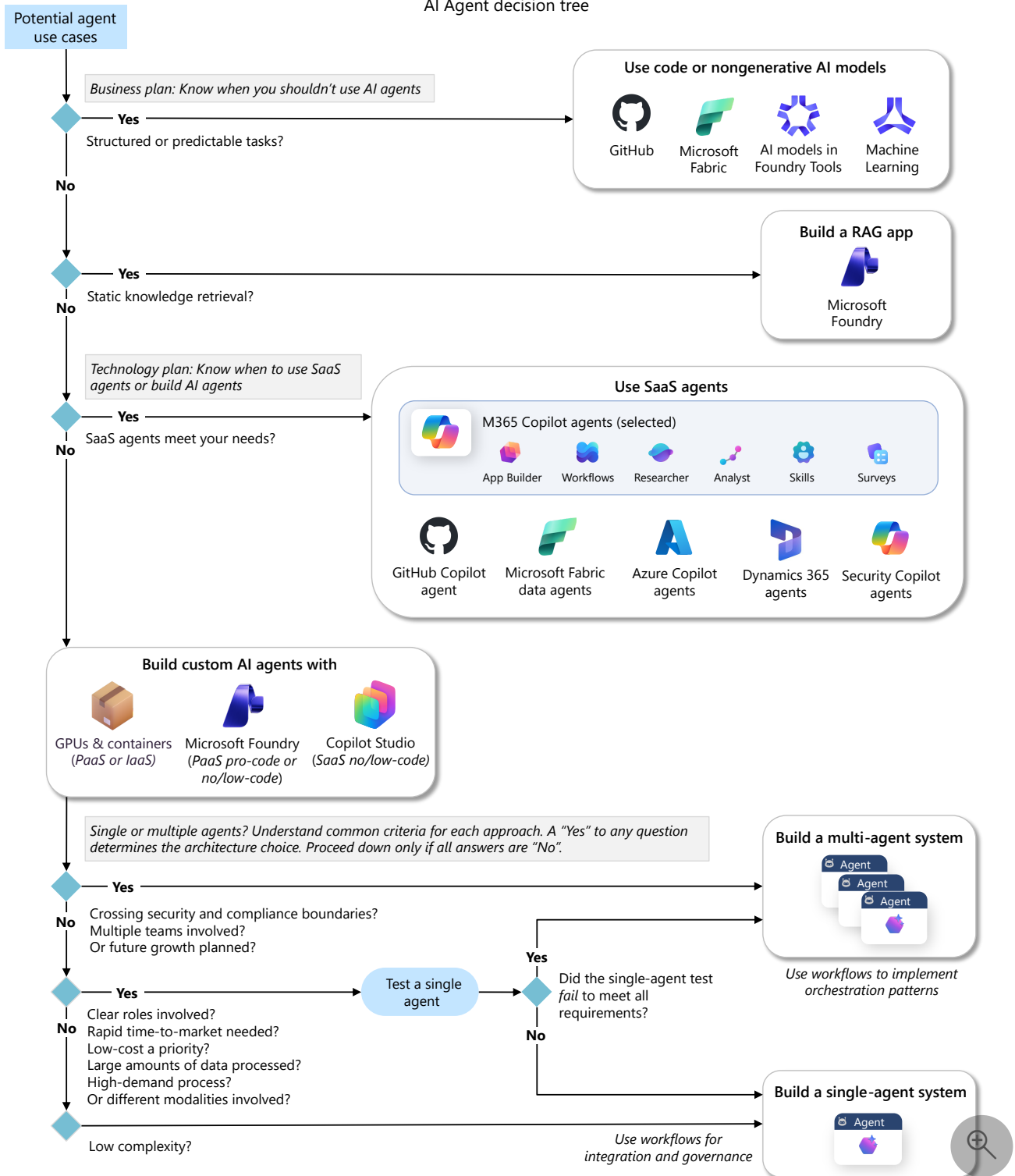
While workload teams need flexibility to make design decisions, organizations benefit from having a standard set of expectations. These expectations help guide how many agents are created and what will need to be governed and maintained over time. To ensure clarity, here are the key definitions:

- **Single-agent systems** consolidate all logic into a single agent. This approach simplifies implementation, reduces operational overhead, and offers a more predictable execution model.
- **Multi-agent systems** divide responsibilities across multiple specialized agents. This enables modularity, clearer separation of concerns, and improved scalability but requires additional coordination and orchestration.

AI Agent decision tree

The AI agent decision tree (see *Figure 2*) helps you determine whether to begin with a multi-agent system, run a single-agent test, or default to a single-agent design. The sections that follow explain every criterion in detail.

AI Agent decision tree



When to start with a multi-agent system

Multi-agent systems deploy two or more agents for distinct tasks within a single business process. This architecture enables different orchestration patterns and specialization. The coordination between agents introduces latency at each handoff point and requires explicit

state management between components. Organizations should **start** multi-agent architecture only when specific criteria mandate separation, such as:

1. **Crossing security and compliance boundaries.** Build multiple agents when regulations or policies mandate strict data isolation. Different security classifications need independent processing environments that single agents can't provide. This least-privilege design limits the blast radius of security incidents by containing breaches within individual agent boundaries. Financial services often require one agent to prepare transactions while another validates them, enforcing separation of duties through architecture.
2. **Multiple teams involved.** Adopt multi-agent designs when distinct teams manage separate knowledge areas. Independent development cycles benefit from decoupled architectures where teams maintain domain-specific agents. Each team uses its specialized expertise and data sources without waiting for other teams. This alignment mirrors organizational structure and enables parallel development. Teams deploy updates independently while explicit interfaces between domains simplify governance and reduce integration risk.
3. **Future growth planned.** Choose modular multi-agent design when the solution roadmap includes diverse features, data sources, or business units. Monolithic agents become unmaintainable as responsibilities expand beyond their original scope. Separating concerns early prevents massive refactoring that disrupts operations. Solutions spanning more than three to five distinct functions benefit from this architecture. Individual agents can modernize independently without affecting the entire system, reducing upgrade risk and enabling incremental improvements.

Multi-agent orchestration and workflows

Multi-agent systems require workflows to implement [orchestration patterns](#) documented in the Azure Architecture Center. Manual chaining of agents creates brittle connections that fail unpredictably. Workflows provide structured coordination that ensures reliable agent interaction. Here are some benefits of using workflows with multi-agent systems:

 Expand table

Workflow capability	Purpose
Coordination	Controls how agents interact through parallel, sequential, or conditional execution patterns
State management	Maintains context across agent boundaries to preserve conversation flow and data integrity

Workflow capability	Purpose
Branching logic	Routes requests to appropriate agents based on conditions, enabling escalation from chatbots to specialized agents or human support
Transparency	Provides visibility into information flow for debugging and compliance auditing

See [Orchestration strategy](#) for technology-specific implementation options.

Multi-agent system trade-offs

Each agent interaction requires protocol design, error handling, and state synchronization. Every component needs separate prompt engineering, monitoring infrastructure, and debugging capabilities. Security surfaces expand through extra credential management and data transit points between agents. Cost structures increase because each agent processes redundant context and communication overhead multiplies with agent count. Latency accumulates at each handoff point, potentially degrading user experience.

When to test a single agent system first

If your use cases don't meet the criteria for multi-agent systems, you should generally start by testing them with a single agent. Validating key assumptions early is critical before selecting the best architecture for a given scenario. While multi-agent architectures are sometimes necessary, they are often chosen based on untested assumptions about complexity or performance. For use cases that meet the following criteria, begin with a single-agent prototype to establish baseline capabilities. Transition to a multi-agent architecture only when testing reveals limitations that cannot be resolved through single-agent optimization.

- 1. Clear roles involved.** Don't assume role separation requires multiple agents. Distinct roles (such as planner, reviewer, executor) might suggest multiple agents, but they don't automatically justify a multi-agent architecture. Often, a single agent using persona switching, conditional prompting, and context-aware policies, can satisfy role-based behavior without added orchestration. Prototype with a single agent to validate assumptions and measure whether role emulation (persona prompts, tool permissioning, and context gating) achieves the required outcomes. Move to multi-agent only when single-agent testing shows limitations you cannot remediate via prompting, retrieval improvements, or policy controls.
- 2. Rapid time-to-market needed.** Single-agent prototypes enable faster validation, faster iteration, and earlier user feedback. Multi-agent systems add coordination logic, communication protocols, and workflow orchestration, which slows early development

and complicates testing. Use a single agent to prove value quickly before investing in multi-agent coordination.

3. **Low-cost a priority.** Single-agent designs reduce token usage and API calls by maintaining context within one entity. Validate cost benefits through prototyping before adding orchestration overhead. Multi-agent systems multiply expenses through redundant context processing and inter-agent communication that can exceed budget constraints.
4. **Large amounts of data processed.** Validate whether a single agent can accurately operate within available context windows while handling the full data scope. Many scalability issues stem from retrieval design, not architecture. For example, teams can adjust chunking and indexing, tailor passage selection to the query, and reduce unnecessary context to fit within the model's window. Only move to multi-agent when testing shows persistent accuracy or latency degradation despite larger context windows, model upgrades, caching, reranking, and prompt/chain optimization.
5. **High-demand process.** Measure throughput and latency with single agents under production-like loads. Multi-agent architectures provide value only when parallelization delivers measurable performance gains. Coordination overhead can negate concurrency benefits in many scenarios, making single agents more efficient.
6. **Different modalities involved.** Start with multimodal models that handle text, images, and other formats within one agent. Use specialized agents only when specific modalities require distinct optimization that general models can't provide. Complex image analysis or real-time audio processing sometimes justifies dedicated agents with specialized resources.

When to use a single agent system

Single-agent architectures consolidate logic, context, and tool execution into one entity. This consolidation reduces design complexity, simplifies implementation, and streamlines governance. Organizations can focus on business value rather than orchestration mechanics. Single agents provide the most efficient starting point for **low complexity** use cases.

1. **Well-defined problem domains.** Choose single agents when workflows follow predictable patterns within bounded contexts. This approach keeps systems maintainable and accelerates development cycles. FAQ bots answering from specific knowledge bases or assistants executing fixed API sequences represent typical single-agent scenarios.
2. **Operational efficiency matters.** Single agents eliminate inter-agent communication protocols that introduce latency and failure points. Debugging becomes straightforward

when all logic resides in one place. Maintenance teams can trace issues without navigating complex agent interactions or distributed logs.

Single-agent workflows

Single agents often respond directly to user requests without orchestration. Workflows provide essential structure for reliability and enterprise integration even with single agents.

- **Repeatability.** Workflows execute consistent tasks across inputs without manual intervention. Nightly batch summarization or scheduled report generation requires workflow automation around single agents.
- **System integration.** Route agent outputs to downstream systems through workflow connectors. Workflows trigger actions including sending results to SharePoint, posting notifications to Teams, or writing data to enterprise databases.
- **Governance and compliance.** Implement logging, approval gates, and audit trails through workflow capabilities. These controls satisfy regulatory requirements and provide operational visibility into agent decisions.
- **Human review.** Insert checkpoints where people validate agent outputs before downstream execution. This human-in-the-loop pattern maintains quality while preserving automation benefits.

See [Orchestration strategy](#) for technology-specific workflows implementation options.

Single-agent system trade-offs

Context length limits restrict information volume that agents process simultaneously. Broad functionality requirements complicate least-privilege security because single agents need permissions for all potential actions. Complex domains can overwhelm single agents, leading to decreased accuracy or increased response times as context grows.

Decision framework

 Expand table

Approach	Use when	Skip prototyping	First step
Single agent	Domain remains narrow, unified context required, speed priority, cost constraints exist	Yes, proceed when scope stays simple and delivery urgency dominates	Deploy single agent with required tools and iterate based on usage

Approach	Use when	Skip prototyping	First step
Comparative prototype	Architecture decision unclear, need evidence for context handling, role separation, or performance	No, prototyping provides decision evidence	Execute controlled tests comparing architectures against defined success metrics
Multi-agent	Hard boundaries exist for security, compliance, or organizational separation, guaranteed multi-domain scaling required	Yes, proceed when requirements mandate architectural separation	Design isolated agents with scoped access and explicit interface contracts

Next step

Process to build AI agents

Last updated on 12/10/2025

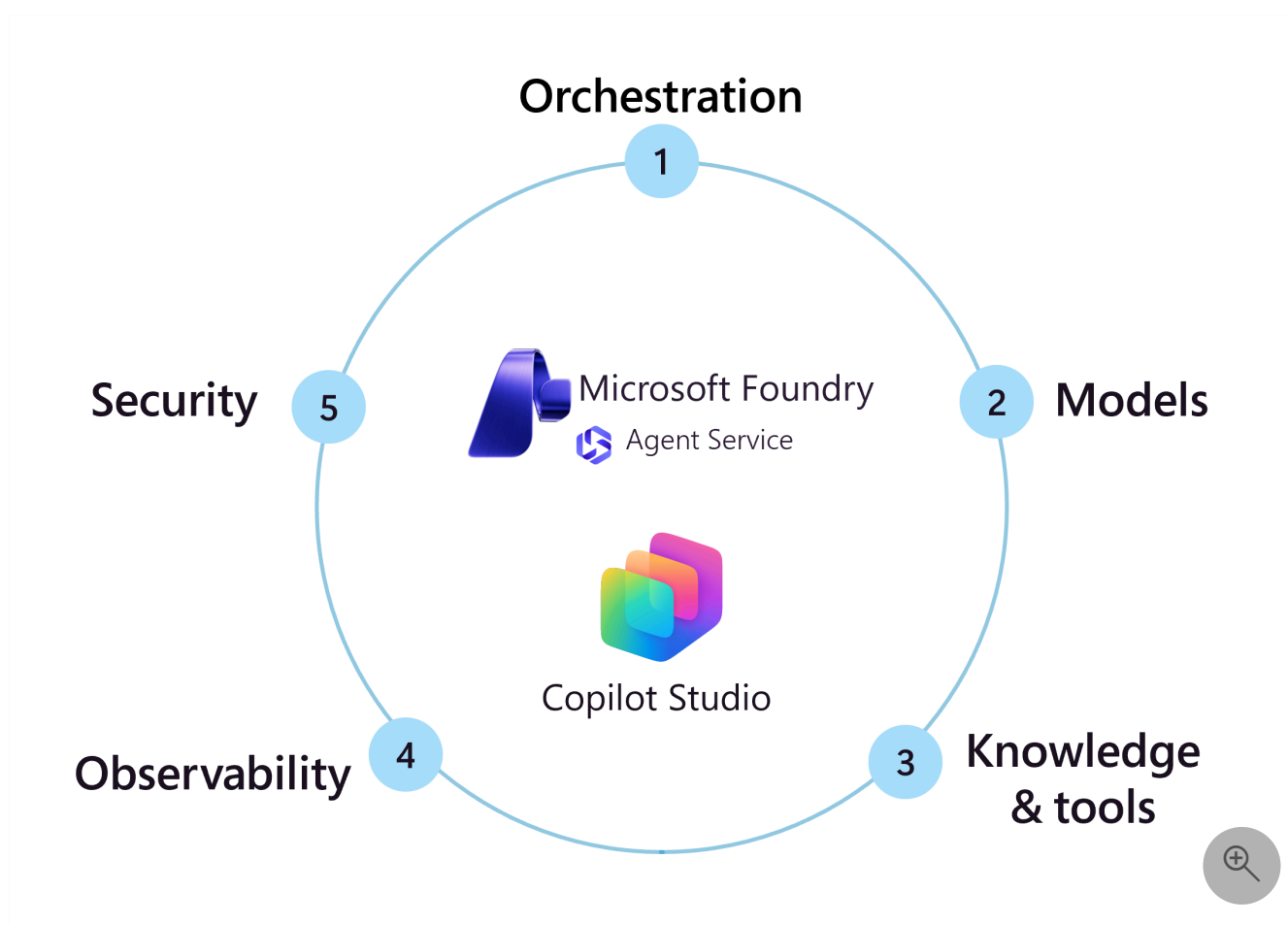
Process to build agents across your organization

This article outlines the standardized process that teams must follow to build AI agents consistently across the organization. This guidance focuses on the governance and structural decisions required to support development teams, rather than the technical implementation of individual workloads. Understanding this process is the final component of the **Build agents** phase in the AI agent adoption process (see *figure 1*).



Figure 1. Microsoft's AI agent adoption process.

Without a unified development process, organizations can experience uncontrolled agent sprawl, inconsistent architectures, and critical security gaps. These problems accumulate as technical debt and operational risk and can prevent the organization from scaling AI adoption effectively. Workload teams must integrate best practices from the Well-Architected Framework's [AI workload](#) guidance.



1. Agent orchestration

Strategic orchestration decisions determine how agents coordinate work, integrate with existing systems, and scale across the enterprise. These choices affect development velocity, operational complexity, and the organization's ability to maintain control as agent adoption grows.

Define the agent charter

Create governance artifacts that document agent boundaries and business alignment. The charter serves as the authoritative reference for what the agent system accomplishes and what it must avoid. This documentation prevents scope creep, ensures compliance teams understand system boundaries, and provides clear accountability when issues arise. Include system responsibilities that map to specific business objectives, agent roles that prevent functional overlap, and prohibited actions that establish compliance guardrails. Without clear charters, agents evolve beyond their intended scope, creating security vulnerabilities and regulatory exposure.

Orchestration strategy

Orchestration determines how agents coordinate, make decisions, and execute tasks. Organizations must define approved orchestration patterns to balance development velocity with operational control. Without standardized strategies, teams could build incompatible systems that are difficult to monitor, debug, and scale.

- 1. Select an orchestration approach.** Choose between managed orchestration and code-first agent frameworks based on organizational capability and control requirements. Managed orchestration, such as the Foundry Agent Service, accelerates deployment and provides built-in security but limits customization. Code-first frameworks, like Microsoft Agent Framework or LangGraph, offer granular control and multicloud flexibility but require significant engineering investment and ongoing maintenance. Define criteria for when teams should use each approach to ensure consistency.
- 2. Define orchestration patterns.** Establish standards for multi-agent coordination. Decide whether to chain agents sequentially or implement parallel processing patterns. Sequential chaining simplifies debugging and provides clear accountability but increases latency. Parallel processing improves response times but requires sophisticated coordination mechanisms and error handling. See [AI agent orchestration patterns](#) in the Azure Architecture Center for architectural decision guides.

3. **Implement deterministic workflows.** Enforce the use of deterministic workflows for critical business logic. Use **workflows** in Foundry and Microsoft Agent Framework to constrain agent behavior within fixed paths rather than relying solely on probabilistic model decisions. Multi-agent systems require workflows to control hand-offs and state management. Review workflow decision guidance for [single agents](#) and [multi-agent systems](#).

Microsoft facilitation:

 **Foundry:** Use [prompt-based agents](#) for rapid development and [workflows](#) for visual, low-code design suitable for rapid prototyping. Use [hosted agents](#) to deploy custom agent frameworks on managed infrastructure.

 **Microsoft Agent Framework:** Use [workflows](#) to implement orchestration patterns in code.

 **Copilot Studio:** [Set up agent flows](#) for multi-agent systems with distinct roles and approval checkpoints. [Configure agent behavior](#) by orchestrating with generative AI.

Agent instructions

Instructions function as the operational logic for AI agents interpret intent and execute tasks. Treating instructions as configuration code, rather than ad-hoc text, enables version control, peer review, and rollback when issues arise.

1. **Standardize instruction architecture.** Consistent structures reduce behavioral drift and ensure compliance. Structure instructions to explicitly define:

 [Expand table](#)

Instruction component	Purpose	Example
Identity and tone	How the agent presents itself to users	Professional, concise, helpful assistant
Scope and boundaries	What the agent must accomplish and avoid	Do not access payroll systems; focus on customer support only
Tool mandates	Specific requirements for tool usage	Use Knowledge Search tool for all answers; state "I don't know" if not found
Citation rules	Requirements for transparency and sourcing	Every answer must provide citations from source material

2. **Enforce structured output formats.** Use formats like JSON or XML for agent outputs to improve interoperability with downstream systems. Structured outputs reduce parsing

errors in multi-agent orchestrations and enable programmatic validation of agent responses before they reach the user.

3. **Implement version control and validation.** Store instructions in source control repositories to maintain an audit trail of changes. Integrate automated validation steps where evaluation scripts or secondary agents verify that updates adhere to safety guidelines and formatting standards before deployment.

Microsoft facilitation:



Foundry: See [System message design](#). Use the [A2A tool](#) and [Foundry playground](#). Test and refine behavior using [agent evaluators](#).

2. Agent model selection strategy

Model selection directly affects cost, performance, and compliance across your agent fleet. Organizations that default to the largest available models face unnecessary costs and quota limitations that constrain scaling.

1. **Match model capabilities to task complexity.** Smaller, optimized models handle routine tasks like summarization efficiently while controlling costs and reducing latency. Reserve premium models for scenarios requiring complex reasoning or multi-step analysis where the value justifies higher costs. This tiered approach prevents budget overruns while ensuring critical workloads receive appropriate resources.
2. **Enforce quota and cost governance.** High-end models impose stricter rate limits that can interrupt service during peak loads. Diversify model usage across workloads to prevent single points of failure. Implement cost allocation tags to track spending by department and use case, enabling data-driven decisions about model investments.
3. **Mandate validation prior to deployment.** Small-scale testing with representative queries documents performance trade-offs and validates compliance with governance standards. This validation prevents costly mistakes where teams deploy expensive models for simple tasks or undersize models for complex requirements.

Microsoft facilitation:



Foundry: Use [model catalog](#) and evaluate model options using model leaderboards to compare performance and cost trade-offs. For productivity and action agents, use the [Model Router](#) to dynamically optimize costs while maintaining quality. Review models' [quotas and limits](#).



Copilot Studio: [Choose a primary AI model](#) or [bring your own model](#) from Foundry.

3. Agent knowledge and tools

Data access and tool permissions determine agent reliability, compliance posture, and operational risk. These policies establish boundaries that prevent unauthorized actions while enabling agents to deliver value. Follow all [data governance and compliance policies](#).

Agent knowledge

Segment knowledge by role to minimize data exposure. Multi-agent systems require strict data boundaries where each agent accesses only information necessary for its function. This segmentation reduces attack surfaces and simplifies compliance audits by creating clear data access patterns. When agents connect to data sources, ensure retrieval occurs only from governed repositories with proper filtering and access controls. Review the [Data architecture for AI agents](#) to align integration strategies with security requirements.

Microsoft facilitation:



Foundry: Connect agents to [Foundry IQ](#) to combine multiple data sources behind an agentic retrieval engine while maintaining governance controls.



Copilot Studio: Use [knowledge sources](#) to connect agents to approved information repositories and [connectors](#) to integrate with enterprise systems securely.

Agent tools

Tools extend agent capabilities from passive information retrieval to active system engagement. This transition introduces operational risk, as agents can now modify data, trigger workflows, and interact with external APIs. Establishing a rigorous governance framework for tool usage ensures that agents operate within defined safety boundaries and adhere to security standards.

1. **Define explicit tool boundaries.** Agents interacting with external systems require strict governance to prevent unauthorized modifications. Specify in the agent's system message exactly when a tool should be invoked and mandate human-in-the-loop confirmation for high-impact actions like database writes or financial transactions. This control prevents autonomous errors and ensures accountability for state-changing operations.
2. **Isolate integration endpoints.** Expose system capabilities through narrowly scoped APIs rather than direct database or system access. Enforce strict input validation on these endpoints to reject malformed data generated by the model before it reaches backend

systems. Apply the principle of least privilege to authentication tokens, ensuring the agent holds only the permissions necessary for the specific task.

3. **Validate actions in isolated environments.** Test agent tool use in sandboxed environments that mirror production but cannot impact live data. Verify that the agent handles API failures or unexpected payloads gracefully without exposing system internals to the user. Configure logging to attribute every action to the specific agent instance and conversation ID, creating a traceable audit trail for debugging and compliance.

Microsoft facilitation:



Foundry: Use [tools](#) and [tool best practices](#). Use an [AI gateway](#) for tool governance.



Copilot Studio: [Add tools](#) and [triggers](#).

Agent memory

Agent memory transforms stateless model inference into stateful business processes. Across the enterprise, memory requirements vary from transient session context to persistent historical recall. Without a unified strategy, ad-hoc memory implementations create vectors for data leakage, privacy violations, and unmanaged storage growth. Organizations must establish governance models that treat conversation history as sensitive enterprise data.

1. **Align persistence models with agent function.** Differentiate memory architecture based on the agent's role. Use ephemeral, session-based memory for transactional agents to minimize data retention risks. Implement long-term, persistent memory only for advisory agents where historical context is critical for personalization or complex reasoning. This segmentation prevents unnecessary data accumulation and aligns storage costs with business value.
2. **Externalize state storage.** Decouple memory from the inference engine. Store conversation history and context in managed data systems where you can apply enterprise security controls, backup policies, and access auditing. This approach ensures data remains under organizational control regardless of the underlying model and enables the application of standard compliance policies to agent interactions.
3. **Enforce strict isolation boundaries.** Prevent context leakage between users, tenants, and agent domains. Ensure that memory retrieval mechanisms strictly respect the authenticated user's identity and permissions. In multi-agent architectures, explicitly define which memory segments can be shared between agents to prevent privilege escalation or data spillage across boundaries.
4. **Automate retention and disposal.** Apply data lifecycle policies to agent memory storage. Configure automated deletion for conversation logs based on regulatory requirements

and business utility. Implement mechanisms that allow users to view and delete their specific memory history to ensure compliance with privacy regulations.

Microsoft facilitation:



Foundry: Use [memory](#) to store conversations. For your own storage, see [standard agent setup](#), [Azure Cosmos DB](#), and [Azure Cosmos DB integration](#).



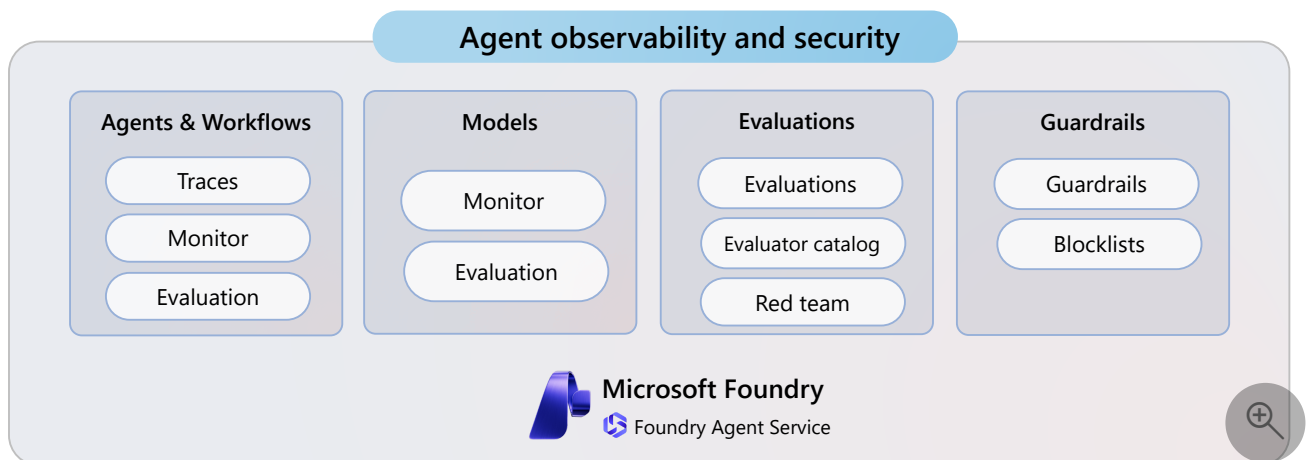
Microsoft Agent Framework: Use [serialization and deserialization](#) of thread state and [agent memory](#).



Copilot Studio: [Analyze agent effectiveness](#) to identify optimization opportunities. Review [user questions by theme](#) to discover patterns and refine responses.

4. Agent observability

Observability provides the visibility needed to maintain reliable agent operations, optimize performance, and ensure compliance with organizational standards. Early implementation of monitoring and telemetry enables proactive issue detection and continuous improvement.



- 1. Implement tracing.** Configure traces that capture execution paths, decision points, and interaction patterns. This visibility enables rapid diagnosis when agents produce unexpected results or experience performance degradation. Establish baselines for latency, cost per interaction, and success rates. Alert teams when metrics deviate from expected ranges to enable investigation before users notice issues.
- 2. Standardize evaluation frameworks.** Create shared evaluations that measure quality, safety, and reliability consistently. Store evaluations in a central catalog to ensure all teams apply the same standards. Integrate evaluations into CI/CD pipelines to catch issues before production deployment. This systematic approach prevents quality drift as agents evolve and new versions deploy.

3. **Extend security testing.** AI agents face unique threats that conventional security testing might miss. Implement dedicated AI red teaming that tests for prompt injection vulnerabilities, attempts to extract training data or system prompts, and validates guardrails against adversarial inputs. Regular security assessments ensure agents remain resilient as attack techniques evolve.

Microsoft facilitation:



Foundry: Use the [Microsoft Foundry Control Plane](#) for unified visibility and governance across your agent fleet.

- **Agents and workflows:** Configure [traces](#) for execution visibility. Configure [monitoring](#) for Foundry instances and all projects. Use [agent evaluators](#) to measure quality. See [Agent Monitoring Dashboard](#).
- **Models:** [Monitor models](#) and use [evaluations](#) to measure AI response quality.
- **Evaluations:** Build and store evaluations in the [evaluator catalog](#). Have teams use [continuous evaluation](#). Integrate evaluations into CI/CD pipelines using [GitHub Actions](#) or [Azure DevOps](#). Use dedicated [AI Red Teaming Agent](#) to scan applications.



Copilot Studio: Use [analytics](#) and [connect to Azure Application Insights](#) for centralized telemetry. [Create test sets for evaluations](#), [Security and governance guidance](#), [automatic security scans](#), and verify [agent runtime protection status](#).

5. Agent security

Security controls embedded throughout the agent lifecycle prevent harmful outputs, protect against attacks, and ensure compliance with organizational standards. These safeguards operate across multiple layers to create defense in depth.

1. **Deploy guardrails at multiple intervention points.** Effective guardrails operate throughout the interaction lifecycle rather than at a single checkpoint. Filter user inputs to block malicious prompts or sensitive data before processing. Validate tool calls to prevent injection attacks when agents interact with external systems. Inspect tool responses for compliance and safety before agents process the information. Apply content moderation and plagiarism checks to final outputs before delivery to users. This layered approach ensures no single failure point compromises the entire system.
2. **Maintain centralized blocklists for consistent enforcement.** Blocklists prevent prohibited terms, sensitive patterns, or unsafe content from entering or leaving the system. Centralize blocklist management to ensure all agents and models apply consistent standards. Update lists regularly based on emerging threats, new compliance

requirements, and lessons learned from incidents. Combine static blocklists with dynamic detection methods to achieve comprehensive coverage while maintaining performance.

Microsoft facilitation:



Foundry: See [Guardrails and control overview](#). Apply guardrails at different [intervention points](#). Use an [AI Gateway](#) to control model endpoints and enforce secure access. Use Microsoft Defender for Cloud's [AI protection](#) and [Gain application and end-user context for AI alerts](#).

Next step

Operate AI agents

Last updated on 12/03/2025

Manage AI agents across your organization

This article provides guidance on integrating AI agents into business workflows and managing their lifecycle from deployment to retirement. Operational excellence ensures that agents deliver consistent value while maintaining security, compliance, and cost efficiency at scale. Learning how to **Manage agents** is the final step in the AI agent adoption process (*see figure 1*).



Figure 1. Microsoft's AI agent adoption process.

Integrating agents into daily operations requires an approach that balances adoption with standardization. You must establish standardized patterns for rollout, monitoring, and maintenance to prevent technical debt and operational fragmentation. Without these controls, organizations face shadow AI proliferation, unpredictable budget overruns, and the accumulation of unused agents that expand the attack surface. This guidance helps you transition agents from development pilots into reliable, managed enterprise assets.

Integrate AI agents

Integration places agents where work happens, whether automating existing tasks or enabling new capabilities. Successful integration requires phased deployment strategies and standardized development patterns that accelerate adoption while maintaining operational control.

Business integration for AI agents

Integrating AI agents requires embedding these tools into daily business operations, not just technical deployment. Successful integration aligns agent capabilities with human workflows to enhance productivity without disrupting established processes. Organizations that treat integration as a purely technical exercise often face low adoption rates and fail to realize the expected return on investment.

1. **Embed agents into existing workflows.** Position agents within the applications and communication channels employees already use, such as Microsoft Teams or custom internal portals. Requiring users to switch contexts to access an agent increases friction and reduces adoption. Seamless integration ensures that agents augment human capabilities at the point of need rather than existing as isolated tools.

2. **Drive adoption through change management.** Implement comprehensive training and communication plans to prepare the workforce for AI collaboration. Employees need guidance on effective prompting, understanding agent limitations, and integrating AI outputs into their decision-making processes. A structured change management approach builds trust and competence, transforming potential resistance into active engagement.
3. **Validate value through phased expansion.** Introduce agents to specific business units or high-impact use cases before a broad organizational rollout. This targeted approach allows you to measure actual business impact, gather qualitative feedback, and refine agent behaviors based on real-world interactions. Expanding based on proven value rather than technical readiness ensures that the agent evolves in alignment with business priorities.

Microsoft facilitation:



Foundry: Use [cost analysis dashboards](#) to monitor token consumption and identify usage patterns.



Copilot Studio: Use [analytics](#) to track engagement and [conversation metrics](#).

Standardization of AI agents

Standardization accelerates adoption and ensures consistent governance, while continuous feedback mechanisms prevent agent capabilities from drifting away from user needs. Without these patterns, organizations face fragmented development efforts and agents that fail to evolve with business requirements.

1. **Develop reusable architectural templates.** Convert proven architectures, prompts, and integration patterns into standardized assets that accelerate development. This approach reduces development time and prevents teams from reinventing solutions for common problems like retrieval-augmented generation (RAG). Templates ensure every deployment inherits required security controls, logging standards, and performance monitoring configurations automatically.
2. **Implement continuous improvement cycles.** Agents require ongoing refinement to remain effective as business needs and data sources evolve. Capture user feedback and operational data to drive iterative improvements rather than treating deployment as a one-time event. Maintain a prioritized backlog of unresolved queries and feature requests to guide development efforts toward high-impact issues that block value delivery.

Microsoft facilitation:



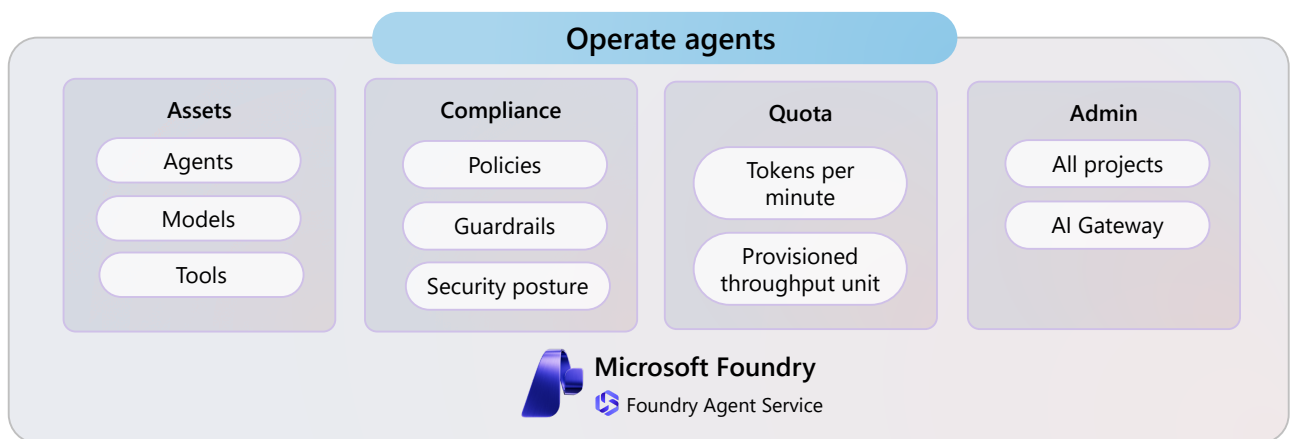
Foundry: See [deployment guidance](#), [rollout guidance](#), [Bicep](#), and [Terraform](#) resources.



Copilot Studio: Use [agent builder templates](#) to accelerate integration and ensure consistency across deployments.

Operate AI agents

Management ensures operational excellence as agents scale across the organization. Without centralized oversight and active lifecycle management, organizations face shadow AI proliferation, budget overruns, and security vulnerabilities. Operations teams must transition from deployment focus to continuous management that maintains governance effectiveness and cost optimization.



Asset lifecycle management for AI agents

Lifecycle management ensures that the AI agent estate remains secure, cost-effective, and aligned with business goals as it evolves. The challenge lies in standardizing operations across disparate teams to prevent fragmentation. Without a unified approach to lifecycle management, organizations risk accumulating technical debt, sustaining unused agents, and losing visibility into operational risks.

- 1. Conduct regular estate audits.** Use the [inventory established during governance](#) to maintain visibility of all active deployments. Identify agents that remain deployed but unused, as these dormant assets create security risks and consume quota allocations. Establish quarterly reviews to retire agents that no longer deliver business value. Regular auditing prevents technical debt accumulation and ensures resources align with current business priorities.
- 2. Centralize project administration.** Use a unified control plane to manage your entire agent fleet rather than administering projects individually. This approach enables consistent policy enforcement, credential rotation, and access control updates across all deployments. Central administration reduces the operational burden on teams while

improving security posture through standardized configurations. Fragmented management across isolated projects creates security gaps that become unmanageable as the agent estate grows.

3. **Systematize cost optimization.** Monitor usage patterns continuously to identify inefficiencies without disrupting service quality. Implement governance controls, including token caps and rate limiting, to prevent runaway costs. Reduce consumption by shortening system prompts, summarizing conversation history, and caching frequent responses. Route deterministic tasks to rule-based logic instead of premium language models to maximize efficiency.
4. **Align availability with criticality.** Configure high availability settings based on workload importance to optimize costs while ensuring resilience. Mission-critical agents require redundancy and failover capabilities that internal, noncritical agents don't. This differentiated approach prevents overspending on unnecessary infrastructure while protecting essential business functions from service disruptions.

Microsoft facilitation:



Foundry: [Optimize model cost and performance](#). Use the [AI Gateway](#) to enforce global token caps and quotas. Monitor consumption with [cost analysis dashboards](#) and configure [autoscaling](#) to manage idle resources. Use the [Microsoft Foundry Control Plane](#) for unified visibility and governance. See [High availability and resiliency](#) and [disaster recovery](#).



Copilot Studio: Use [analytics](#) to monitor token usage per session and identify high-cost skills. Implement [Azure API Management](#) to enforce throttling and logging policies.

Compliance and security of AI agents

Continuous compliance ensures that AI agents remain secure and aligned with regulatory standards as they evolve. Without ongoing monitoring, static security configurations quickly become obsolete, exposing the organization to data leakage, regulatory fines, and reputational damage. Operations teams must automate policy enforcement and integrate agent security into broader organizational defense strategies.

1. **Automate policy enforcement.** Schedule automated scans to detect configuration drift and policy violations before they become incidents. Verify that agents continue to meet data residency, retention, and privacy requirements as regulations change. The [governance policies](#) and [security standards](#) established during development require ongoing enforcement, as static configurations become vulnerabilities over time. Automated enforcement reduces the manual review burden while improving consistency across the agent estate.

2. **Integrate with security operations.** Route agent security alerts to your Security Operations Center (SOC) for rapid incident response. Treat jailbreak attempts, data leakage indicators, and anomalous behavior patterns with the same urgency as traditional security threats. This integration ensures agent-specific risks receive appropriate attention and response resources. Security teams must understand agent-specific threat patterns and response procedures to handle incidents effectively.
3. **Validate through red teaming.** Conduct adversarial testing periodically, especially after major model updates or architectural changes. Red team exercises reveal vulnerabilities that automated scanning might miss. Focus testing on new attack vectors and evolving threat patterns to ensure defenses remain effective against current risks. Regular testing validates that security controls function as intended and identifies gaps before attackers exploit them.

Microsoft facilitation:



Foundry: Use [Azure Policy](#) to enforce compliance standards. Integrate with [Microsoft Defender for Cloud](#) for threat protection.



Copilot Studio: Use [Data Loss Prevention \(DLP\) policies](#) to control data flow and connector usage.

Quota optimization of AI agents

Quota optimization ensures that agent workloads receive necessary resources while preventing cost overruns from inefficient usage. Without active quota management, organizations risk service interruptions for high-value agents due to resource exhaustion by lower-priority tasks. Operations teams must balance availability requirements against budget constraints through dynamic allocation strategies.

1. **Review consumption trends monthly.** Analyze usage data regularly to identify inefficient resource utilization that wastes budget. The [cost tracking mechanisms](#) established during governance provide visibility into spending patterns. Agents using premium models for simple tasks waste resources that could support higher-value activities. Monthly reviews reveal optimization opportunities and prevent gradual cost creep from becoming budget crises.
2. **Adjust quotas based on business priorities.** Use consumption data to refine quota allocations dynamically based on actual usage patterns and business needs. Critical workloads warrant higher quotas to ensure uninterrupted service, while experimental or low-value activities need stricter limits to control costs. Dynamic adjustment ensures resources flow to initiatives that deliver the greatest business impact, whereas fixed allocations become inefficient as usage patterns evolve.

3. **Optimize model selection for tasks.** Evaluate whether smaller, more cost-effective models can handle routine tasks without sacrificing quality. Reserve premium models for complex reasoning and nuanced language understanding that simpler models can't achieve. This tiered approach maximizes return on investment while maintaining service quality where it matters most. Regular benchmarking ensures model selection remains optimal as capabilities and costs change.

Microsoft facilitation:



Foundry: [Plan and manage costs](#). Use the [management center](#) to administer quotas and monitor usage against budgets.



Copilot Studio: [Review usage and message allocation](#) to manage consumption and optimize licensing.

Administration of AI agents

Administration centralizes control over the AI agent ecosystem to ensure consistency, security, and operational efficiency. Without unified administration, organizations face fragmented governance, inconsistent policy enforcement, and difficulty responding to widespread incidents. Effective administration requires visibility into all deployments and the ability to enforce controls.

1. **Maintain comprehensive project visibility.** Keep an up-to-date inventory of all AI agent projects and deployments across the organization. Visibility forms the foundation of effective governance, as you can't secure or optimize resources you can't see. Regular inventory updates ensure new deployments receive appropriate oversight from creation, preventing shadow deployments from creating unmanaged risk.
2. **Implement an AI gateway.** Route all AI traffic through a managed gateway to create a unified control point for policy enforcement. This abstraction layer enables centralized monitoring, security controls, and traffic management without modifying individual agent implementations. The gateway provides consistent logging, throttling, and access control across heterogeneous agent deployments, simplifying governance and reducing complexity.
3. **Enforce granular controls.** Configure token limits and usage quotas at the project or individual agent level based on business requirements. Implement pause and resume capabilities for external agents and tools to respond quickly to security incidents or operational issues. These controls prevent runaway costs and enable rapid containment when problems arise, ensuring resources are used efficiently while maintaining operational flexibility.

Microsoft facilitation:



Foundry: View all Foundry projects in the Admin dashboard. Enable [AI Gateway](#), powered by Azure API Management, to govern AI model deployments, tools, and agents in your Foundry resources.

Last updated on 12/04/2025